

Advanced Machine Learning

Lisa Giacobazzi

1 Machine Learning

OBIETTIVO: Un sistema deve migliorare la sua performance P in una data Task T attraverso l'esperienza E .

Vogliamo che i sistemi siano adattivi, robusti e fault tolerant.

1. Ottenere degli esempi di quello che vogliamo
2. Raccogliere dati
3. Stimare una funzione che da un input trovi l'output $\leftarrow$ Supervised Learning
4. Trovare dei pattern all'interno dei dati $\leftarrow$ Unsupervised Learning

1.1 Supervised Learning

Nel supervised Learning l'algoritmo impara da degli esempi labeled (x_i, y_i) e stima una funzione che predice y su dei dati non labeled.

Possiamo fare Binary Classification, Multiclass Classification e Regressione

Queste si distinguono perchè hanno rispettivamente:

- 2 opzioni di output (es. spam / non spam)
- n opzioni finite (es. classificazione di animali)
- output continuo (es. prezzo di una casa)

Esempi pratici:

- Binary classification: "Questa email è spam?"
- Multiclass classification: "Qual è la specie di questo fiore?"
- Regressione: "Quanto costerà questa casa in base ai metri quadri?"

1.2 Unsupervised Learning

Analizza dei dati non labeled e identifica dei pattern nascosti. L'output è la **struttura latente** che emerge dai dati (es. clustering, riduzione di dimensionalità).

Esempi pratici:

- Clustering: segmentazione di clienti in gruppi simili
- PCA (Principal Component Analysis): riduzione del numero di variabili in un dataset

1.3 How to deal with data

1. Osservare i dati:
Prima di tutto, raccogliamo dati sul fenomeno che vogliamo studiare o modellare.
2. Riassumere la loro incertezza:
I dati non sono mai perfetti: c'è rumore, mancano valori, ci possono essere errori. Quindi si cerca di modellare l'incertezza, ad esempio stimando probabilità, intervalli di confidenza, distribuzioni.
3. Cercare dei constraint e invariances (Prior Knowledge):
In questa fase si cerca di trovare regolarità nei dati cercando anche di integrare conoscenza che già abbiamo
4. Trovare un modello che li spieghi:
Identificare quale conoscenza abbiamo sul mondo e usarla per costruire il modello, cerchiamo di costruire un modello matematico o computazionale che spieghi i dati, tenendo conto dell'incertezza e dei vincoli trovati prima.

5. Rappresentare i dati:
Rappresentare le feature e modello di classificazione - posso decidere ad esempio se diminuire il numero di features tramite la dimensionality reduction
6. Sfruttare il calcolo:
trovare una soluzione tramite tecniche di ottimizzazione e calcolo numerico.
7. Evitare il data bias:
garantire che i dati siano rappresentativi e privi di distorsioni sistematiche.

1.4 Discriminative VS Generative Learning

L'apprendimento **supervisionato** stima la relazione tra input e output. I due modelli sono quello discriminativo e quello generativo.

- Discriminativo:
Stima $\mathbb{P}(Y|X)$, ossia predice direttamente l'etichetta
Esempi: Logistic Regression, SVM
- Generativo:
Stima $\mathbb{P}(X, Y)$, il joint model di X e Y e ne ricava $\mathbb{P}(Y|X)$ tramite Bayes
Esempio: Naive Bayes

Esempio:

- Discriminativo: un classificatore che distingue email spam da non-spam osservando solo le probabilità condizionate.
- Generativo: un modello che prova a capire come sono fatte le email spam e le email normali (modella tutto).

1.5 Estimator

Un **estimator** è una funzione o metodo che stima un parametro sconosciuto a partire da dati osservati. Hai dei dati, ma non conosci alcuni parametri del fenomeno che li ha generati (es. la media vera, la relazione tra variabili, ecc), un estimator prova a indovinare questi valori a partire dai dati.

- In statistica: stima media, varianza, ecc.
- In ML: è un modello che può essere addestrato su dei dati e che vi fa previsioni (es. regressione lineare).

La linear regression è uno stimatore in quanto trova la linea retta che "riassume" una nuvola di punti.

1.5.1 Bias e Varianza di uno stimatore:

- **Bias**: differenza tra il valore atteso dello stimatore e il valore vero.
Il bias misura quanto in media lo stimatore sbaglia rispetto al valore vero che dovrebbe stimare. E' la differenza tra l'average estimate e il valore vero. L'aspettativa dello stimatore è la media teorica dello stimatore su infiniti campioni.
Estimator of Bernoulli mean: unbiased
Estimator of Gaussian Mean: unbiased
Estimator of Gaussian variance: biased! Perché usa la stima della media invece della media vera, perdendo quindi un grado di libertà! Dobbiamo quindi dividere per n-1 invece che per n

$$\frac{1}{n-1} \sum (X_i - \bar{X})^2$$

- **Varianza**: quanto lo stimatore cambia al variare dei dati.

La varianza misura quanto lo stimatore oscilla da un campione all'altro.

L'estimatore ideale è unbiased e con una varianza piccola.

- **Standard Error:** radice della varianza; indica quanto ci si aspetta che la stima fluttui tra i campioni.

1.6 Mean Squared Error (MSE)

$$MSE = Bias^2 + Varianza$$

Il Mean Squared Error è uno dei metodi più efficaci per valutare la qualità di uno stimatore in quanto combina il bias e la varianza in un'unica metrica.

Misura quanto lo stimatore si discosta dal valore vero, in media, al quadrato. Al quadrato perché penalizza di più gli errori grandi e mantiene tutto positivo

Anche se uno stimatore è biasato, può avere MSE più basso di uno non biasato, se ha varianza molto più piccola.

1.7 Bias-Variance Decomposition

È un modo per scomporre l'errore totale di un modello in tre parti:

1. **Bias quadrato:**
È l'errore sistematico, quanto la media delle previsioni del modello si discosta dalla verità. Succede quando il modello è troppo semplice per rappresentare il fenomeno.
Esempio: usare una retta per approssimare un fenomeno curvo.
2. **Varianza:**
È l'errore dovuto a quanto il modello cambia se gli dai dati leggermente diversi. Modelli troppo complessi tendono a variare molto, cioè a "overfittare".
Esempio: un modello che segue troppo i punti del training set e generalizza male.
3. **Rumore:**
È l'errore intrinseco nei dati, che non può essere eliminato da nessun modello.

Aumentare la complessità del modello tende a ridurre il bias ma aumentare la varianza, e viceversa.

—

2 Deep Learning e Funzionamento Generale

Cose principali nell'apprendimento supervisionato (valide anche per deep learning):

1. **Training Data:** insieme di esempi etichettati $(x, y) \in X \times Y$
2. **Build a Model:** si costruisce una funzione $h : X \rightarrow Y$ che cerca di cogliere la relazione tra input e output. Il modello impara.
3. **Make Predictions:** si usa h per predire l'etichetta di nuovi input

Ma cosa differenzia ML e DL?

1. Nel Deep Learning avremo un **Feature Learning automatico** invece del feature engineering
2. **Modelli diversi:** reti neurali invece che decision trees, per esempio
3. I dati non dovranno più essere necessariamente numerici o ben strutturati, ma si potranno usare anche immagini, audio e testi grezzi

2.1 Loss e True Risk

La **loss** misura l'errore tra la predizione $h(x)$ e la vera etichetta $y = f(x)$.

La loss è la funzione obiettivo che si minimizza durante l'addestramento.

E' una funzione che misura l'errore tra la predizione assegnata (fatta dal modello) e la correct labeling function, che è una funzione ideale che simuliamo con le etichette nei dati di input.

Si calcola durante la fase di training per aggiornare i pesi (Il bias invece si calcola dopo)

Esempio di **loss 0-1**:

$$L(h(x), y) = \begin{cases} 1 & \text{se } h(x) \neq y \\ 0 & \text{se } h(x) = y \end{cases}$$

La loss è una misura empirica.

Il **true risk** rappresenta l'**errore medio** del modello su tutta la distribuzione $\mathcal{D}$ dei dati.

E' la probabilità che NON venga predetta la label corretta in un data point random di $\mathcal{D}$

$$R_{\mathcal{D}}(h) := \mathbb{P}[h(X) \neq f(X)] = \mathbb{P}[h(X) \neq Y]$$

Il true risk di un learner h è quindi il valore atteso della loss, misura la capacità del modello di generalizzare

In pratica: *quanto spesso sbaglia il modello nel mondo reale.*

2.2 Empirical Risk

L'empirical risk diventa una stima necessaria quando la distribuzione $\mathcal{D}$ è ignota. Se conoscessimo $\mathcal{D}$ potremmo calcolare direttamente il true risk del modello h , ma nella pratica non conosciamo mai veramente $\mathcal{D}$.

Quando non conosciamo la distribuzione, usiamo i dati di training — che sono stati campionati da $\mathcal{D}$ — per stimare il rischio empirico, ossia l'errore sul dataset di training:

$R_S(h)$ = rischio empirico: ci dice **quanto bene il modello ha imparato dai dati di training**.

L'empirical risk è la miglior proxy che abbiamo del true risk quando non possiamo accedere a tutta la distribuzione dei dati reali, cosa che succede praticamente sempre. Non garantisce che generalizzi bene su dati nuovi.

Il rischio empirico è la media della loss sul training set.

2.3 Bayes Classifier e Bayes Risk

Il **Bayes Classifier** opera con l'assunzione che l'input space sia discreto.

Il suo obiettivo è trovare una funzione che minimizzi la probabilità di misclassificazione, ossia il true risk. Vuole quindi minimizzare $\mathbb{P}[h(X) \neq Y]$, che può essere espressa come la sommatoria su tutti gli input x del prodotto $\mathbb{P}(h(X) \neq Y|X = x)\mathbb{P}(X = x)$

Per ogni specifica x settiamo $h(x)$ in modo che minimizzi la probabilità condizionata $\mathbb{P}(h(X) \neq Y|X = x)$, cosa che corrisponde a massimizzare la probabilità $\mathbb{P}(h(X) = Y|X = x)$

Il **Bayes Classifier** $h^*(x)$ è **ottimale**, in quanto assegna ad ogni x l'etichetta y che massimizza la probabilità a posteriori:

$$h^*(x) = \arg \max_{y \in Y} \mathbb{P}[Y = y \mid X = x]$$

Il **Bayes Risk** $R_{\mathcal{D}}(h^*)$ è l'errore minimo teoricamente raggiungibile sotto la distribuzione $\mathcal{D}$. È il true risk più basso ottenibile.

Nessun algoritmo può andare meglio del Bayes Risk. È il limite inferiore dell'errore.

2.4 PAC Learning (Probably Approximately Correct)

2.4.1 No Free Lunch Theorem

Senza fare assunzioni sul problema o sulla distribuzione dei dati, nessun algoritmo può garantire di apprendere bene in generale.

Implicazione: ogni algoritmo può funzionare bene **solo su alcune classi di problemi** — non su tutti.

Nella teoria dell'apprendimento, si vorrebbe idealmente che l'algoritmo trovi sempre una funzione esattamente corretta, su tutti i dati. Nella pratica questo è **irrealizzabile**.

Quindi si fa una **relaxation** (rilassamento della garanzia):

Tipo di relax	Significato	Parametro
Approximately correct	L'ipotesi h può avere un piccolo errore	ϵ
Probably correct	Possiamo tollerare un po' di incertezza nel successo dell'apprendimento	δ

In apprendimento automatico esistono quindi diversi livelli di garanzie. Alcuni algoritmi sono solo approximately correct (errore basso, ma senza garanzie probabilistiche). Altri sono only probably correct (corretti solo con alta probabilità, ma magari con errore alto). Gli algoritmi più solidi sono PAC: garantiscono errore basso con alta probabilità. Infine, molti algoritmi moderni funzionano bene in pratica, ma **non offrono nessuna garanzia teorica**

PAC combina entrambe le garanzie: **Probably (con confidenza $1 - \delta$) approximately (errore $\leq \epsilon$) correct**.

Non possiamo imparare perfettamente in generale, ma possiamo imparare con alta probabilità (probably) una funzione quasi corretta (approximately correct).

Nel PAC learning il learner opera senza avere una prior knowledge sulla distribuzione $\mathcal{D}$ o sul classificatore Bayesiano, ma deve avere:

- La classe di ipotesi ha capacità limitata,
- I dati sono i.i.d. da una distribuzione fissa,
- L'algoritmo ha errore empirico basso.

Il learner avrà due parametri key

- ϵ (**accuracy parameter**): tolleranza massima sull'errore
- δ (**confidence parameter**): probabilità massima di fallimento del learner

Il learner può avere un dataset di training con $m(\epsilon, \delta)$ samples, la grandezza di questo dataset dipende quindi dai requisiti di accuratezza e confidenza.

L'obiettivo è di produrre una ipotesi h tale che

$$\mathbb{P}[R_{\mathcal{D}}(h) \leq \varepsilon] \geq 1 - \delta$$

- δ rappresenta la **probabilità di fallimento** (cioè che $R_{\mathcal{D}}(h) > \varepsilon$)
- $1 - \delta$ è la **confidence**, cioè la probabilità che h sia abbastanza buono

Con probabilità almeno 95% (cioè $1 - \delta = 0,95$), l'ipotesi h ha errore minore di 5% (cioè $\varepsilon = 0,05$).

Il numero di esempi richiesti $m(\varepsilon, \delta)$ **non può dipendere** da $\mathcal{D}$ né dal Bayes classifier. Il learner opera **senza conoscenza della distribuzione** o della funzione target.

Il PAC learning fornisce una **base teorica solida** per l'apprendimento supervisionato in condizioni realistiche, dove:

- non conosciamo la distribuzione dei dati,
- vogliamo garantire generalizzazione,
- e siamo disposti a tollerare piccoli errori con alta probabilità.

DEFINIZIONE:

Una classe delle ipotesi H è **PAC learnable** se esiste una funzione $m_{\mathcal{H}} : (0, 1)^2 \rightarrow \mathbb{N}$ e un algoritmo di apprendimento con le seguenti proprietà:

1. Per ogni $\epsilon, \delta \in (0, 1)$
2. Per ogni distribuzione $\mathcal{D}$ su $X \times Y$ tale che la *realizability assumption* tiene in H quando l'algoritmo ha $m > m_{\mathcal{H}}(\epsilon, \delta)$ esempi i.i.d. generati da $\mathcal{D}$, esso ritorna un'ipotesi h tale che, con probabilità almeno $1 - \delta$,

$$R_{\mathcal{D}}(h) \leq \epsilon$$

2.5 PAC learning con Realizability Assumption

La realizability assumption semplifica il PAC learning introducendo una condizione:

Esiste un'ipotesi $h^* \in H$ tale che $R_{\mathcal{D}}(h^*) = 0$
 Ossia, c'è una ipotesi perfetta nello spazio delle ipotesi

Nella classificazione binaria questa assunzione assicura che il learner conosca H e che essa sia abbastanza espressiva da includere la true target function.

La realizability assumption è facoltativa. Quando viene fatta, il PAC learning è detto “realizzabile” e si semplificano molte dimostrazioni. Ma esistono anche forme **agnostiche**, dove questa assunzione **non vale**.

Quando la realizability assumption tiene, è possibile avere un algoritmo straightforward:

1. Il learner ottiene un training dataset S con m samples
2. L'algoritmo identifica ogni ipotesi $h \in H$ che soddisfa $R_S(h) = 0$, ossia che hanno il rischio empirico a 0.

Questo approccio è un esempio del Empirical Risk Minimization (ERM - Sezione 2.8), ossia il learner seleziona l'ipotesi che minimizza il rischio empirico

2.6 Infinite Hypothesis Classes

A volte lavoriamo con **classi di ipotesi infinite** (es. rette, soglie reali, intervalli). Ma in pratica:

1. **Vincoli computazionali**: i computer usano precisione finita (es. 32 bit)
2. **Approssimazione finita**: ogni parametro può assumere solo un numero finito di valori → la classe diventa effettivamente finita
3. **Euristica campioni**: serve un numero di esempi almeno 10 volte il numero di parametri per garantire buona generalizzazione

Conclusione: anche se è infinita in teoria, possiamo trattarla come finita in pratica grazie alla discretizzazione.

Questo ci permette di applicare il PAC learning anche in scenari apparentemente complicati.

2.6.1 Extra: Frasi chiave da ricordare

- $R_{\mathcal{D}}(h)$: true risk = errore atteso sul mondo reale
- $R_S(h)$: empirical risk = errore sui dati di training
- $R_{\mathcal{D}}(h^*)$: Bayes risk = errore minimo possibile
- ε : quanto errore accetto
- δ : quanto rischio sono disposto a correre
- PAC: con almeno $1 - \delta$ di probabilità, $R_{\mathcal{D}}(h) \leq \varepsilon$

2.7 Overfitting

L'overfitting (che corrisponde a una varianza alta) accade quando un modello performa estremamente bene nei dati di training ma non riesce a generalizzare su dati mai visti → la sua performance con i dati reali è deludente.

In alcune situazioni l'overfitting può essere evitato: se la classe delle ipotesi è finita e il dataset contiene un numero sufficiente di esempi, è più difficile che ci sia overfitting → questo evidenzia la differenza critica tra training error e true error, perchè quando H è troppo grande e il dataset piccolo, il training error può ingannare.

2.7.1 Cross Validation

Nella pratica, normalmente si inizia con un unico dataset, che dividiamo in 3 subset distinti:

- **Training Set**: usato per *allenare* il modello. Serve per costruire una famiglia di modelli (ipotesi) cambiando i parametri.
- **Validation Set**: usato per *scegliere il miglior modello* tra quelli costruiti, cioè l'ipotesi. Si misura l'errore su questi dati per decidere quale modello selezionare. Il validation set non può essere considerato neutro, perchè viene usato per fare molte valutazioni e il modello viene scelto in base a quei dati
- **Test Set**: usato **una sola volta**, alla fine, per stimare il true error (*quanto bene il modello funzionerà su dati mai visti*) del modello. Serve per avere una stima unbiased dell'errore. Il suo utilizzo è essenziale perchè l'errore misurato sul validation set tende a essere biasato negativamente (più ottimistico).

La cross validation serve per valutare il modello **usando solo il training set**, senza intaccare il test set.

K-Fold Cross Validation:

1. Si divide il training set in k blocchi.
2. Si allena su $k - 1$ blocchi e si valida sul rimanente.
3. Si ripete k volte, cambiando blocco di validazione ogni volta.
4. Si calcola la media degli errori di validazione.

L'utilizzo del test set è essenziale perchè l'errore misurato sul validation set tende a essere biasato negativamente, ma solo se il test set è sufficientemente grande, altrimenti diventa unreliable nella stima del true error.

Bisogna ricordare che quando il miglioramento è minimo, potrebbe essere dato da fluttuazioni randomiche più che da un effettivo miglioramento della performance.

2.7.2 Model Selection

Quando si sceglie un modello bisogna tenere presenti i seguenti principi:

Trade-off tra Complessità e Varianza: Aumentare la complessità del modello può migliorare l'adattamento ai dati, ma anche aumentare la **varianza** e il rischio di **overfitting**.

Modelli Potenti Richiedono Più Dati: I modelli molto espressivi (es. reti neurali profonde) hanno bisogno di dataset ampi per generalizzare bene.

Esclusività del Test Set: Il test set non deve essere usato per ottimizzare o selezionare il modello. Serve **solo per la valutazione finale**.

2.8 Empirical Risk Minimization

L'ERM è una strategia di apprendimento supervisionato, un framework che cerca il modello con errore più basso sul training set.

L'ERM base si limita a minimizzare l'**errore sui dati di training**, ovvero il *rischio empirico*. Questo può portare a **overfitting**, perché il modello si adatta troppo bene ai dati di training.

Per evitare l'overfitting, con la regularization si aggiunge una **penalità alla complessità del modello**, ottenendo una **versione regolarizzata** di ERM.

- Il primo termine è l'errore empirico (come nell'ERM normale),
- Il secondo termine è la **penalizzazione**

2.8.1 Regularization

La regularization è una versione più robusta di ERM, che tiene conto anche della complessità del modello.

La regolarizzazione è una tecnica usata per prevenire l'**overfitting**, dove un modello diventa troppo complesso e impara il noise.

Funziona aggiungendo un termine di penalità alla funzione di loss, in modo da scoraggiare parametri troppo larghi e di promuovere modelli semplici

$$ErroreTotale = ErroreTraining + \lambda \cdot Complessità(h)$$

La λ è un parametro, un numero positivo che **controlla il compromesso** tra “fit” ai dati e semplicità del modello. È come un peso che regola quanto ci fidiamo dei dati rispetto alla nostra preferenza per modelli più semplici.

2.9 Linear Regression

L'analisi della regressione considera il problema della predizione quando lo spazio delle label Y è continuo. In questo contesto misurare la qualità del predittore h usando la predizione dell'errore $\mathbb{P}[h(x) \neq Y]$ non ha senso. Il goal è trovare una h che minimizza l'errore misurato usando una loss function

Loss function:

- Standard choice for classification: 0/1 loss
- Standard choice for regression: squared loss $L(\hat{y}, y) = (\hat{y} - y)^2$

Per minimizzare la loss empirica (quadratica) troviamo l'ERM.

Il **Least Squares (LSQ) Criterion** è il principio su cui si basa la **regressione lineare classica**: minimizzare l'errore quadratico medio sulle predizioni. È un **caso specifico di Empirical Risk Minimization**, dove la loss scelta è il **quadrato dell'errore**.

Il **Least Squares Criterion** è il **criterio complessivo** che somma (o fa la media) di tutte le squared loss sui dati di training.

Un predittore lineare ha forma $\hat{y} = wx + b$, il least squares criterion è un metodo fondamentale nell'analisi della regressione usato per stimare i parametri (b, w) del modello, in modo che la somma delle differenze quadratiche tra i valori osservati e predetti, sia minimizzata. Nel caso della regressione, il goal è trovare la best-fitting line.

2.9.1 Ridge Regression

È una versione con regularization della regressione lineare, in cui **si minimizza la somma della differenza quadratica** e di un **termine di regolarizzazione** che penalizza i pesi grandi e aiuta a prevenire l'overfitting.

$$L(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda |w|^2$$

L'introduzione del termine di regolarizzazione aiuta a controllare l'overfitting soprattutto quando i sample data sono pochi e il feature space è grande.

2.9.2 Polinomial Regression

La regressione polinomiale permette l'apprendimento di predittori non lineari, utilizzando però lo stesso framework della regressione lineare.

Consideriamo un predittore $\hat{y} = b + w_1x + w_2x^2 + \dots + w_mx^m$ anche se la funzione appare non lineare, lo è in termini di coefficienti (b, w) , questo significa che si può ancora applicare il metodo dei minimi quadrati per l'apprendimento del modello. Quando diciamo che un modello è **lineare nei parametri**, intendiamo che l'**output è una combinazione lineare dei pesi**, anche se **non necessariamente delle feature originali**.

Questo approccio funziona sia per input 1D che per input higher dimensional, permettendoci di modellare relazioni complesse mantenendo un processo di ottimizzazione lineare.

—

3 Regression for Classification

Possiamo usare la regressione per predire una categoria invece che un valore facendo un post processing per convertire la regressione lineare a un output binario. In questi casi la soluzione non è più necessariamente ottima.

Può essere usato un threshold per decidere l'appartenenza a una classe `if (y < 0.5) class 1; else class 2`

Un'altra soluzione potrebbe essere modificare la loss per minimizzare i suoi valori categorici direttamente. La loss nella regressione lineare minimizza l'errore quadratico (least squares), non vuole massimizzare $\mathbb{P}(y|x)$.

La linear regression può quindi essere usata per fare classificazione, ma:

1. **Output non interpretabile come probabilità:**

L'output della LiR produce valori da - a + infinito, che non sono interpretabili come probabilità di appartenenza a una classe.

2. **Sensibilità agli outlier:**

Un singolo dato "strano" può far oscillare pesantemente la linea di regressione.

3. **Non ottimizza la funzione di loss adatta alla classificazione:**

In classificazione binaria, vogliamo massimizzare la probabilità di classificazione corretta, **non** minimizzare l'errore quadratico.

Una soluzione migliore sarebbe utilizzare la logistic regression, la quale usa una funzione sigmoide sull'output per comprimerlo tra 0 e 1, e usa una **loss specifica per classificazione**, la **log-loss** (o cross-entropy). In questo modo il valore di $\hat{y}$ può essere interpretato come la probabilità di appartenenza alla classe.

3.1 Support Vector Machine

È un algoritmo di **classificazione supervisionata** che ha l'obiettivo di **trovare il confine migliore** (iperpiano) tra due classi di dati. L'obiettivo è trovare **l'iperpiano che separa le due classi con il margine massimo**.

L'iperpiano in uno spazio 2D, un iperpiano è una **retta**. In 3D, è un **piano**. In generale, è una **superficie lineare** che divide lo spazio in due.

Esempio

Hai due classi di punti: rossi e blu. Vuoi tracciare una **linea che li separa**.

SVM **non sceglie una linea qualsiasi**, ma quella che **massimizza la distanza tra i punti più vicini alla linea e la linea stessa**.

Questa distanza è chiamata **margine**.

L'**iperpiano ottimale** è quello che **massimizza il margine**, i punti più vicini al confine (quelli che lo "sostengono") sono chiamati **support vectors**. Anche se cambi gli altri punti, **solo questi support vectors determinano la soluzione!**

Massimizzare il margine equivale a minimizzare la norma dei pesi, soggetti ai vincoli di separabilità.

Nella Support Vector Machine la loss è definita per massimizzare il margine tra classi, penalizzando i punti misclassified.

3.2 Gradient Descent

E' un algoritmo di ottimizzazione iterativo per trovare i parametri (es. pesi) ottimali che minimizzino la loss.

L'idea è di muoversi dove la funzione diminuisce di più.

Immagina di essere su una montagna, bendato, e vuoi **scendere nella valle più profonda (minimo)**.

- Ti chiedi: “In che direzione la discesa è più ripida?”

- Ti muovi **in direzione opposta al gradiente** (cioè alla salita).

L'aggiornamento dei parametri avviene:

$$w := w - \eta \cdot \nabla L(w)$$

Dove w è il vettore dei pesi, $\eta \cdot \nabla L(w)$ è il gradiente della loss rispetto ai pesi, cioè il vettore di derivate parziali, e η è la learning rate.

Gli stationary point sono punti in cui il gradiente vale 0: $\nabla f(x) = 0$

Se il gradiente è zero significa che ci troviamo in un punto in cui non c'è **direzione in cui la funzione cresce o decresce localmente**. Questo non significa necessariamente che ci troviamo in minimo globale: potremmo trovarci in un minimo locale, un massimo o un punto di sella

3.2.1 Learning Rate

Il **learning rate** è uno degli **iperparametri più importanti** del gradient descent.

E' un **numero positivo** (di solito piccolo) che **controlla quanto velocemente il modello aggiorna i suoi parametri** durante ogni passo del gradient descent.

L'**overshooting** avviene quando la learning rate è troppo alta, il che significa che l'algoritmo fa passi troppo grandi e di conseguenza fa “jump over” del minimo della loss function, portando a oscillazioni o divergenza.

Se è il learning rate è **troppo grande**, si rischia di saltare il minimo e divergere, se è **troppo piccolo**, la convergenza è troppo lenta.

Il learning rate può essere adattivo o seguire una schedule.

1. Follow a schedule: Si diminuisce il learning rate seguendo un parametro di decadimento.
2. Adaptive: Si accumulano i gradienti e si continua a muoversi nella loro direzione. La learning rate sarà direttamente proporzionale a quanto è allineata la sequenza di gradienti. (momentum)

3.2.2 Gradient Descent for Deep Learning

Il gradient descent può essere applicato a problemi non-convex, senza garanzie di ottimità, ma siamo raramente interessati all'ottimo globale, in quanto non è generalizzabile.

Un problema è **non-convesso** quando la funzione obiettivo **non ha una forma a “ciotola” (convessa)**, cioè **può avere più minimi locali e massimi locali**, non un unico minimo globale.

- Convex $\rightarrow$ come una parabola: ha **un solo minimo** $\rightarrow$ facile da ottimizzare

- Non-convex $\rightarrow$ come una montagna russa: **tanti alti e bassi** $\rightarrow$ difficile trovare il minimo “migliore”

Normalmente nel DL ogni parametro viene aggiornato in modo da diminuire la loss $\theta_i \leftarrow \theta_i - \eta \frac{\delta l}{\delta \theta_i}$
 Il gradiente ci dice come modificare il parametro:

1. θ contiene i parametri della neural network, che possono essere milioni
2. La loss può essere non-convex e non differenziabile $\rightarrow$ non si può applicare il gradient descent standard
3. Bisogna considerare la complessità computazionale e la stabilità numerica.

3.2.3 Stochastical Gradient Descent

Usualmente la loss è definita su n esempi di training., quindi serve calcolare il gradiente per ogni termine.

Ci sono quindi due bottlenecks che rendono il gradient descent poco pratico:

1. Il numero di esempi
2. Il numero di parametri

Possiamo quindi calcolare il gradiente per un piccolo sottoinsieme rappresentativo $m < n$ - mini-batch - degli esempi, che viene preso in modo uniforme . Il true gradient è approssimato, ma c'è uno speed-up significativo.

Per ogni iterazione del gradient descent possiamo prendere un nuovo sottoinsieme su cui calcolare il gradiente, quando (a forza di iterare e prendere sottoinsiemi diversi) avremo coperto l'intero training set originario, avremo una epoca

Il costo di aggiornamento è costante indipendentemente dalla grandezza del dataset originario.

Quindi, il gradiente stocastico non si ferma al minimo e le oscillazioni sono date dal rumore indotto dal random sampling.

SGD non dipende dal numero di esempi, cosa che implica una generalizzazione migliore. SGD può trovare un basso valore della loss velocemente, anche se non è un minimo, **proprio questa fluttuazione spesso aiuta a uscire da minimi locali poco buoni.**

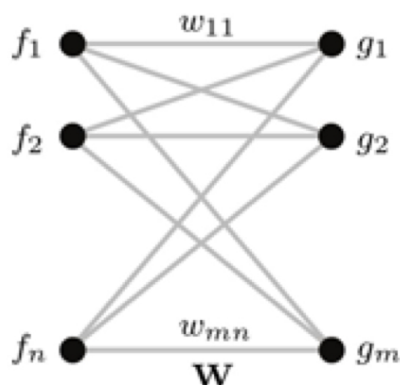
Considerazioni:

1. Bisogna fare dello **shuffling dei training data** per evitare il bias.
 Fare **shuffling dei dati** significa **mescolare casualmente l'ordine degli esempi** nel dataset di training prima di ogni epoca (o prima di ogni suddivisione in mini-batch).
 Quando si usa **Stochastic Gradient Descent (SGD)** o **Mini-Batch Gradient Descent**, se **non mescoli i dati**, potresti introdurre dei **bias nell'ottimizzazione**. Ad esempio:
 - Se i dati sono **ordinati per classe** o per tipologia (es. prima tutti i gatti, poi tutti i cani), i mini-batch saranno **non rappresentativi del dataset intero** $\rightarrow$ il gradiente sarà **distorto**.
 - Lo stesso vale per **dataset ordinati nel tempo**: gradienti troppo simili rischiano di **convergere a soluzioni povere** o poco generalizzabili.
2. Ogni mini-batch può essere processato in parallelo, ma la sua grandezza è limitata dall'hardware.
3. Con dataset grandi ci vogliono più step (iterazioni) per raggiungere la convergenza.
4. Mini-batch piccoli possono offrire un effetto di **regularization**, in quanto portano a una maggiore **varianza nella stima del gradiente** e all'uso di una **piccola learning rate** per mantenere stabilità.

3.3 Neural Network

3.3.1 Single Linear Layer

Una rete neurale semplice è composta da un singolo layer lineare in cui l'output g_ℓ viene calcolato come:



$$g_\ell = \sigma \left(\sum_{\ell'=1}^n f_{\ell'} w_{\ell\ell'} \right), \quad \ell = 1, \dots, m$$

dove:

- $f_{\ell'}$ sono gli input (feature),
- $w_{\ell\ell'}$ sono i pesi della matrice W ,
- σ è una funzione di **attivazione**, ad esempio la **ReLU**: $\sigma(x) = \max(x, 0)$
- $\ell = 1, \dots, m$ indica ogni neurone del layer di output, NON il layer

I **parametri** del layer sono i pesi W , inclusi eventualmente i bias.

Una rete a singolo layer lineare è limitata nella sua capacità di rappresentare funzioni complesse perché non può catturare relazioni non lineari nei dati (a meno di fare feature engineering).

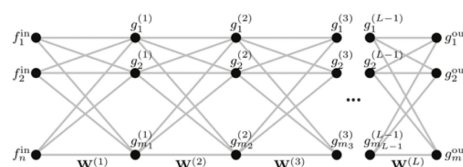
3.3.2 Deep Neural Network Consisting of L Layers

Una **rete neurale profonda (deep neural network)** è composta da una sequenza di layer lineari + attivazione. Ogni layer prende in input l'output del layer precedente:

$$\mathbf{g}^{(k)} = \sigma \left(\mathbf{W}^{(k)} \mathbf{g}^{(k-1)} \right)$$

dove:

- $\mathbf{g}^{(k)}$ è il vettore di output del layer k ,
- $\mathbf{W}^{(k)}$ è la matrice dei pesi del layer k ,



In altre parole, ogni layer fa un prodotto matrice-vettore seguito da una trasformazione non lineare. Questo permette alla rete di apprendere relazioni molto complesse.

3.3.3 Considerazioni sui Deep Networks

Le deep neural network hanno **grande capacità espressiva**, ma presentano anche **difficoltà pratiche**:

1. **Possiamo renderle arbitrariamente complesse**
Aggiungendo più layer e neuroni, la rete può rappresentare funzioni via via più complicate.
2. **Il numero di parametri può essere enorme**
Più layer e più neuroni $\rightarrow$ più pesi da apprendere $\rightarrow$ rischio di overfitting se i dati non sono sufficienti.

3. Sono difficili da ottimizzare

Il problema è **non-convesso** → molti minimi locali, punti di sella → l'ottimizzazione (gradient descent) è difficile.

4. È difficile ottenere una buona generalizzazione

Reti troppo complesse possono adattarsi perfettamente ai dati di training ma generalizzare male (*overfitting*).

4 Il Perceptrone

Il Perceptron è un classificatore supervisionato molto semplice ma estremamente importante dal punto di vista storico e concettuale. Fino ad ora si sono analizzati modelli lineari e con una sola componente, come ad esempio la semplice funzione $f(x) = w \cdot x + b$, capaci di risolvere solo problemi linearmente separabili.

Il perceptrone segue questa stessa idea: è anch'esso un classificatore lineare, ispirato ai neuroni biologici, ma applicato alla classificazione binaria. Proprio per la sua linearità, condivide le stesse limitazioni: non può risolvere problemi non linearmente separabili.

Le caratteristiche principali del perceptrone:

- Gli **input** sono i **valori delle features** (cioè le caratteristiche degli esempi di training)
- Ogni feature ha associato un **peso**, che indica quanto quella feature è rilevante
- Si calcola la **sommatoria pesata** degli input: questa si chiama **attivazione**
- Se l'attivazione è **positiva**, l'output è +1
- Se l'attivazione è **negativa**, l'output è -1

$$\text{activation}_w(x) = \sum_i w_i \cdot x_i = w \cdot x$$

Quindi, l'attivazione è semplicemente il prodotto scalare tra il vettore dei pesi w e l'input x .

4.1 Algoritmo di apprendimento del Perceptron

1. **Inizializza i pesi**: si parte con $w = 0$
2. **Per ogni esempio nel training set**, eseguire i seguenti passi:
 - a. **Classificare** l'esempio usando i pesi attuali:

$$y = \begin{cases} +1 & \text{se } w \cdot x \geq 0 \\ -1 & \text{se } w \cdot x < 0 \end{cases}$$

- b. Se la classificazione è **corretta** (cioè $y = y^*$), **non si fa nulla**
- c. Se invece è **errata**, si aggiorneranno i pesi sommando (o sottraendo se $y^* = -1$) il vettore delle features.

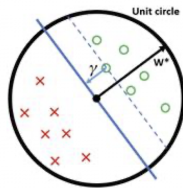
Questo procedimento si ripete **per ogni esempio**, uno alla volta, in modo **iterativo**.

4.1.1 Convergenza e garanzie teoriche

Si può dire che il perceptron sia stato il primo algoritmo con una **forte garanzia formale**

Se un dataset è **linearmente separabile**, il perceptron troverà un iperpiano che lo separa in un numero finito di iterazioni, altrimenti andrà in loop per sempre.

Teorema - limite superiore sugli errori



- Se tutti gli input x_i stanno all'interno della sfera unitaria, ossia il valore massimo dell'input è 1
- Se esiste un iperpiano separatore w^* con norma 1, ossia che il **vettore dei pesi** w che definisce l'iperpiano è **normalizzato**, cioè ha **lunghezza (norma) pari a 1**
- Se γ è la distanza del punto più vicino all'iperpiano separatore, e $\gamma > 0$

→ Il numero **massimo** di errori che il perceptron può commettere è $\frac{1}{\gamma^2}$

→ Se i dati sono ben normalizzati e linearmente separabili, il Perceptron **converge in un numero finito di passi**, e non fa più di $1/\gamma^2$ errori.

4.1.2 Problemi del perceptron

1. **Rumore nei dati**: se i dati non sono separabili linearmente, i pesi possono "oscillare" indefinitamente (*weight thrashing*).
2. **Generalizzazione mediocre**: può trovare soluzioni che separano a malapena le classi.
3. **Overtraining**: l'accuratezza sul training/test set può aumentare e poi diminuire, segno di **overfitting**.

4.2 Dal Perceptron Deterministico a quello Probabilistico

4.2.1 Perceptron Deterministico e Linear Regression

La struttura del Perceptrone è simile a quella della regressione lineare, ma il Perceptrone è usato per classificare, mentre la regressione lineare per prevedere valori continui. Non sono intercambiabili, ma condividono la componente lineare $w^T x + b$

Il Perceptron definisce un **iperpiano** che divide lo spazio delle feature in due regioni, una per la **classe positiva** (+1) e una per la **classe negativa** (-1)

In termini geometrici:

- Se $w \cdot x + b > 0$ allora l'esempio è positivo
- Se $w \cdot x + b < 0$ allora è negativo

In questo modo il Perceptron è in grado di creare una **frontiera di decisione lineare**, ossia un confine tra le classi che è una retta (in 2D), un piano (in 3D) o, più in generale, un iperpiano.

Possiamo stabilire se un punto è classificato correttamente usando la seguente condizione:

$$y_i(w^T x_i) > 0$$

Questa espressione significa che il punto x_i si trova dal **lato corretto dell'iperpiano**. In particolare, se y_i è +1, allora $w^T x_i$ deve essere positivo, e se y_i è -1, allora deve essere negativo.

Questa regola funziona solo se $y_i \in \{-1, +1\}$. Non funzionerebbe se, ad esempio, i target fossero $\{0, +1\}$.

4.2.2 Perceptron Probabilistico e Logistic Regression

Il Perceptron classico prende decisioni **deterministiche**: dato un input, produce un output secco, tipicamente $+1$ o -1 , sulla base del segno di $w \cdot x$.

Tuttavia, in molte applicazioni reali è più utile ottenere una **stima probabilistica**, ossia una misura del grado di confidenza della previsione. Per questo motivo, si passa da una decisione deterministica a una **formulazione probabilistica**.

La **logistic regression** può essere interpretata come una **versione probabilistica del perceptrone**.

Anziché restituire un'etichetta, calcola uno **score** $z = w \cdot x$ e lo trasforma in una probabilità attraverso la **funzione sigmoide**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In questo modo:

- Se z è molto positivo, $\sigma(z) \approx 1 \rightarrow$ alta probabilità di appartenenza alla classe positiva
- Se z è molto negativo, $\sigma(z) \approx 0 \rightarrow$ alta probabilità di appartenenza alla classe negativa

A differenza del perceptrone, che aggiorna i pesi solo quando sbaglia e non minimizza una funzione di loss continua, la logistic regression definisce una vera e propria funzione obiettivo. I parametri w vengono stimati tramite la **maximum likelihood estimation (MLE)**, ossia massimizzando la probabilità che il modello assegni correttamente le etichette osservate.

Questo equivale, dal punto di vista dell'ottimizzazione, a **minimizzare la log-loss** (nota anche come cross-entropy), che penalizza fortemente le classificazioni errate con alta confidenza.

La regressione logistica è un modello lineare: la probabilità è modellata con una sigmoide, che è una funzione non lineare, ma la decision boundary è un iperpiano. Quindi, da sola, può classificare correttamente solo dati linearmente separabili.

4.2.3 Estensione Multiclasse del Perceptron Probabilistico

Nel caso di più di due classi, il perceptrone probabilistico può essere esteso associando a ciascuna classe $y \in \{1, \dots, k\}$ un proprio vettore di pesi w_y .

Dato un input x , si procede così:

- Si calcola uno **score** lineare per ciascuna classe: $z_y = w_y \cdot x$
- Si seleziona la classe con punteggio più alto: $y^* = \arg \max_y z_y$

Per ottenere una previsione probabilistica invece di una scelta secca, si applica la **funzione softmax** agli score.

Softmax trasforma un vettore di score in una distribuzione di probabilità: il punteggio più alto riceve la probabilità maggiore, ma anche gli altri hanno un peso, il tutto somma a 1 ed è positivo.

Questa trasformazione restituisce una distribuzione di probabilità su tutte le classi, permettendo di interpretare $\text{softmax}(z_y)$ come la probabilità che l'input appartenga alla classe y .

Come nella regressione logistica binaria, anche in questo caso i parametri $w_1, \dots, w_k$ vengono stimati tramite **maximum likelihood estimation (MLE)**, massimizzando la probabilità che il modello assegni le etichette corrette ai dati osservati.

4.3 SGD e Perceptron

Si può vedere il Perceptron come un caso particolare di **ottimizzazione con discesa del gradiente stocastica (SGD)**.

- L'ipotesi è: $y = \text{sign}(w^T x)$
- Si può definire una **hinge loss**, che è una funzione che serve a misurare quanto sbaglia il modello, costruita per spingerlo a classificare correttamente e con margine. Questa loss è nulla quando il punto è classificato correttamente con margine, e positiva quando si commette un errore o si è troppo vicini alla frontiera.

La regola di aggiornamento dei pesi nel perceptrone può essere vista come un caso particolare di aggiornamento SGD su questa funzione di loss. Se un esempio è classificato correttamente, non si aggiorna nulla; se invece è classificato male, i pesi vengono modificati secondo:

$$w_{t+1} = w_t + \eta_t y_t x_t$$

dove η_t è il learning rate e si può anche fissare a 1, e $y = \pm 1$, quindi

- Se l'errore è su un esempio positivo: $w_{t+1} = w_t + x$
- Se è su uno negativo: $w_{t+1} = w_t - x$

Quindi la regola di aggiornamento del perceptrone è un caso particolare di SGD, dove la loss è non differenziabile ovunque, ma il gradiente può essere definito a tratti (come con hinge loss), e l'aggiornamento avviene solo quando si sbaglia.

4.4 Modelli Avanzati e Reti Neurali

Nella pratica, molti fenomeni reali sono **altamente non lineari**. Quindi, abbiamo bisogno di modelli di apprendimento che siano in grado di catturarli.

Un buon modello dovrebbe essere il più universale possibile.

Ricetta generale per un modello parametrico:

- Fissare la forma generale del modello
- Ottimizzare i parametri
- Assicurarci che il modello sia gestibile (computationally friendly)

Un esempio semplice di **modello non lineare parametrico** è: $\sigma \circ f(x)$, cioè applicare una **funzione non lineare** (sigmoide) sopra una funzione lineare.

4.4.1 Il Multi-Layer Perceptron (MLP)

Se σ è la funzione logistica, allora abbiamo il modello di **regressione logistica**.

Tuttavia, l'**innovazione** nasce quando si combinano **più strati di trasformazioni lineari e non lineari** (regressione logistica), formando una **struttura gerarchica** in cui ogni livello elabora l'output del precedente. Questo meccanismo a strati multipli, consente di rappresentare anche **funzioni non lineari** e quindi risolvere problemi che non sono linearmente separabili.

Questo porta al concetto di **composizione di funzioni**:

$$\text{output} \leftarrow (\sigma \circ f) \circ (\sigma \circ f) \circ \dots \circ (\sigma \circ f)(x)$$

La funzione logistica:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

è **continua** e introduce **non linearità** nel modello, permettendo di catturare pattern più complessi rispetto ai modelli lineari.

Altre funzioni di attivazione possono essere utilizzate, ad esempio:

- **ReLU**: $\sigma(x) = \max(0, x)$, che ha **gradiente discontinuo** ma garantisce **efficienza computazionale** e **differenziabilità parziale**.

4.4.2 Definizione Formale e Architettura

Chiamiamo la composizione di funzioni lineari e non lineari come:

$$g_\theta(x) = (\sigma \circ f_{\theta_n}) \circ (\sigma \circ f_{\theta_{n-1}}) \circ \dots \circ (\sigma \circ f_{\theta_1})(x)$$

una **rete neurale deep feed-forward** o **Multi-Layer Perceptron (MLP)**.

1. **Parametri distribuiti nei vari strati:**

Ogni strato ha i propri pesi e bias, indicati con W_ℓ e b_ℓ .

2. **Rappresentazioni intermedie:**

L'output di ciascuno strato $x_{\ell+1}$ si ottiene applicando una funzione di attivazione non lineare al risultato della trasformazione lineare:

$$x_{\ell+1} = \sigma_\ell(W_\ell x_\ell + b_\ell)$$

3. **Trucco tecnico sul bias:**

Il bias può essere incorporato nel peso aggiungendo una costante 1 al vettore in input. Questo semplifica l'espressione matematica senza perdere generalità.

4. **Implicazione pratica:**

Ogni funzione f è **lineare rispetto ai parametri**, come nella regressione lineare. Tuttavia, la **non linearità** σ consente al modello di **superare i limiti espressivi** di un approccio puramente lineare.

Ogni riga della matrice dei pesi è chiamata "neurone" o "unità nascosta".

Possiamo avere due interpretazioni:

1. Ogni layer è una **funzione da vettore a vettore**: $\mathbb{R}^p \rightarrow \mathbb{R}^q$
2. Ogni layer ha **q unità** che agiscono in parallelo. Ogni **unità agisce come una funzione scalare** $\mathbb{R}^p \rightarrow \mathbb{R}$

Il **layer di output** determina il **codominio** (cioè l'intervallo dei valori di output) della rete neurale.

La forma generale è:

$$y = (\sigma \circ f) \circ (\sigma \circ f) \circ \dots \circ (\sigma \circ f)(x)$$

Se è la funzione sigmoide logistica, l'intera rete calcola:

$$\mathbb{R}^p \rightarrow (0, 1)^q$$

Tuttavia, **per generalità**, è comune aggiungere un **layer lineare finale**:

$$y = f \circ (\sigma \circ f) \circ (\sigma \circ f) \circ \dots \circ (\sigma \circ f)(x)$$

Questo permette alla rete di mappare: $\mathbb{R}^p \rightarrow \mathbb{R}^q$

Cioè, anche verso valori reali, non solo tra 0 e 1.

4.4.3 Deep ReLU Networks

Aggiungendo un layer lineare all'output $y = f \circ \sigma(\dots)(x)$ si può interpretare l'output come una **combinazione di** ridge functions.

Una ridge function è una funzione non lineare $(\sigma(w^T x + b))$ ma costante lungo certe direzioni.

In una rete a 2 layer, con una ridge function con attivazione ReLU: $\sigma(x) = \max(0, x)$ otteniamo una **funzione piecewise-linear**.

Questo significa che lo spazio degli input è **suddiviso in regioni**, e in ogni regione la funzione appresa è lineare, ma **diversa da zona a zona**.

La ReLU è utile perchè:

1. Introduce **non linearità** nel modello.
2. Mantiene la **semplicità computazionale**.
3. Evita problemi di **saturazione**: le funzioni come la sigmoide hanno gradienti molto piccoli per valori estremi (problema del **vanishing gradient**), mentre la ReLU ha gradiente costante pari a 1 per input positivi.
4. **Sparsità**: se un'unità riceve un input negativo, il suo output è 0 $\rightarrow$ molte unità "si spengono" in ogni layer. Questo rende il modello **più efficiente e regolare**, come un sistema che si auto-seleziona le unità più attive.

Dying ReLU: se un neurone riceve sempre input negativo, non si aggiorna più (gradiente 0).

Quali funzioni può rappresentare un MLP?

Se la funzione di attivazione σ è **sigmoidale**, si ha il seguente risultato:

Teorema di Approssimazione Universale (Universal Approximation Theorem):

Esiste una rete neurale (con un solo layer nascosto e funzione sigmoide) che può **imitare qualsiasi funzione continua** con **precisione arbitraria**, purché usiamo **abbastanza neuroni**.

$\rightarrow$ **le reti neurali sono universalmente espressive**

Se q (cioè il numero di neuroni) è abbastanza grande, **l'errore di training può essere reso arbitrariamente piccolo**.

Nota: il teorema vale anche per altre funzioni di attivazione, come la **ReLU**, se sono **continue, limitate localmente, non polinomiali**.

Empiricamente si osserva che:

- Usare **reti più larghe** (cioè con q grande)
- Combinato con la **discesa del gradiente**

porta spesso a **ottime approssimazioni**, anche per funzioni molto complesse.

4.4.4 Training di un MLP: Ottimizzazione e Generalizzazione

Durante il training, una rete MLP riceve **coppie input-output** x_1, y_1 , e il suo output è ottenuto **applicando una sequenza di trasformazioni** con parametri θ (cioè i pesi e i bias):

Il processo di apprendimento consiste nel **minimizzare la funzione di perdita** su tutto il dataset. Una funzione molto comune è la **Mean Squared Error (MSE)**:

Minimizzare questa loss significa **modificare i pesi** θ per migliorare la qualità dell'output.

Ma attenzione: **la MSE non è convessa rispetto a θ** in generale, quindi l'ottimizzazione è difficile.

Casi speciali in cui la MSE è convessa:

1. Rete a **singolo layer** senza funzioni di attivazione $\rightarrow$ **regressione lineare**
2. Rete a **singolo layer** con attivazione sigmoide $\rightarrow$ **regressione logistica**

4.5 Autodifferenziazione e Computational Graph

Per addestrare le reti neurali, si usano algoritmi simili alla discesa del gradiente, quindi si vuole calcolare il **gradiente** della funzione di loss rispetto ai pesi della rete, in modo efficiente.

Poiché la funzione è composta da molti passaggi intermedi (attivazioni, somma pesata, normalizzazioni...), è necessario applicare la **regola della catena** in modo efficiente e sistematico.

$$f(x) = g(h(x)) \rightarrow \frac{df}{dx} = \frac{dg}{dh} \cdot \frac{dh}{dx}$$

Per farlo, costruiamo un **grafo computazionale (computational graph)**. È un grafo diretto aciclico (DAG) in cui:

- ogni nodo è un'operazione (es. somma, log, moltiplicazione)
- ogni arco rappresenta una dipendenza tra operazioni
- i nodi interni sono variabili intermedie

Questo grafo descrive come viene calcolata la funzione: dall'input all'output, passo per passo.

Esempio

Considera la funzione:

$$f(x) = \log x + \sqrt{\log x}$$

Costruiamo il grafo computazionale:

$$\begin{aligned}y &= \log x \\z &= \sqrt{y} \\f &= y + z\end{aligned}$$

Il grafo ha tre operazioni collegate. Ora vogliamo calcolare $\frac{df}{dx}$, cioè la derivata della funzione composta.

4.5.1 Forward Mode (Autodifferenziazione in avanti)

Calcoliamo le derivate **da input a output**, seguendo i passaggi della funzione.

$$f(x) = y + z \rightarrow \frac{df}{dx} = \frac{dy}{dx} + \frac{dz}{dx} = \frac{dy}{dx} + \frac{dz}{dy} \cdot \frac{dy}{dx} = \frac{1}{x} + \frac{1}{2\sqrt{y}} \cdot \frac{dy}{dx} = \frac{1}{x} + \frac{1}{2x\sqrt{\log x}}$$

In generale, nella forward mode:

- si parte dagli input e si propaga la derivata avanti
- si ottiene la derivata rispetto a un solo input alla volta

È efficiente quando il numero di **input è piccolo** (pochi parametri), ma ci sono tanti output.

4.5.2 Reverse Mode (Backpropagation)

Questa è la modalità più usata nel deep learning.

Funziona al contrario: prima si calcolano tutti i valori in avanti (forward pass), poi si risale il grafo (backward pass) calcolando le derivate con la regola della catena ****al contrario****.

Vantaggi:

- Calcola tutte le derivate rispetto agli input **in un solo passaggio**
- È efficiente quando c'è **un solo output** (es. la loss) e **tanti input** (milioni di pesi)

Perfetto per l'addestramento di reti neurali.

4.5.3 Conclusione

Il motivo per cui le librerie come `PyTorch` e `TensorFlow` sono efficienti è che:

- costruiscono automaticamente il grafo computazionale mentre scrivi il codice
- applicano reverse-mode autodifferentiation per calcolare i gradienti

Questo è ciò che rende possibile il training di reti con milioni di parametri in tempi ragionevoli.

5 Reti Deep Feed-Forward: Potenza e Limiti

Le **reti deep feed-forward** (note anche come multilayer perceptron, MLP) sono reti neurali in cui l'informazione fluisce in un'unica direzione: dagli input verso gli output, attraversando una serie di strati (layer) intermedi. In teoria, queste reti sono **probabilmente universali**, ovvero capaci di approssimare qualsiasi funzione continua con sufficiente accuratezza.

Tuttavia, questa universalità teorica porta con sé una serie di **problemi pratici**:

- Possiamo costruirle con una **complessità arbitraria**, aumentando indefinitamente il numero di strati e nodi.
- Il **numero di parametri** da apprendere può diventare estremamente elevato, rendendo difficile l'ottimizzazione.
- Sono **molto difficili da addestrare**: l'ottimizzazione può facilmente bloccarsi in minimi locali o saddle point.
- È **complesso ottenere una buona generalizzazione**: le reti rischiano di imparare a memoria i dati di training senza saper generalizzare a dati nuovi.

5.1 Il Ruolo dei Priori

Per affrontare questi limiti, è utile **introdurre dei priori** (o "conoscenze a priori"). In termini semplici, un *prior* rappresenta una supposizione o vincolo che inseriamo nel modello prima di vedere i dati. Serve a **guidare l'apprendimento** e a **semplificare il problema**, rendendo più robusto il modello.

I priori ci aiutano a definire meglio che tipo di modello costruire, basandoci sulla natura del problema che vogliamo risolvere.

Idealmente, cerchiamo **priori "universali"**, cioè **indipendenti dal task specifico**. Tuttavia, tali priori **devono essere presenti nei dati stessi**. Non possiamo inventarli: se i dati non contengono struttura, non possiamo aspettarci che il modello ne benefici.

5.2 Priori Strutturali nei Dati

Spesso i dati reali **non sono completamente caotici**, ma possiedono una certa **struttura interna**. In particolare, le immagini (ma anche i segnali audio, i testi, ecc.) mostrano:

- **Pattern ripetuti**: le stesse forme o texture possono apparire in più punti.
- **Composizionalità**: le strutture complesse possono essere scomposte in parti più semplici.
- **Località**: le informazioni rilevanti tendono ad essere locali, cioè concentrate in piccole regioni.

Queste proprietà ci permettono di **definire priori strutturali**, ovvero assunzioni sulla struttura dei dati che possiamo incorporare nei nostri modelli per renderli più efficienti e robusti.

Se i dati hanno una qualche forma di regolarità, possiamo e dobbiamo sfruttarla inserendola nei modelli tramite i priori.

5.2.1 Autosimilarità nei Dati

Un concetto chiave in molti domini, specialmente nelle immagini, è quello di **autosimilarità**:

I dati tendono a essere auto-simili nel dominio, cioè lo stesso tipo di pattern può ripetersi in posizioni diverse.

Questa proprietà giustifica, ad esempio, l'uso delle **convoluzioni** nelle CNN (Convolutional Neural Networks), dove si assume che un pattern appreso in una parte dell'immagine possa essere utile anche altrove.

5.3 Invarianza alla Traslazione

Una proprietà fondamentale che ci aiuta a definire modelli più intelligenti è l'**invarianza alla traslazione**. Essa afferma che:

Se trasliamo un oggetto in un'immagine (cioè lo spostiamo), il suo contenuto non cambia.

Dal punto di vista matematico, questa idea si esprime definendo un operatore di traslazione lineare T_v rispetto a un vettore $v \in R_2$:

$$T_v f(x) = f(x - v)$$

Questo formalismo ci permette di costruire modelli (come le CNN) che **non dipendono dalla posizione assoluta** di un oggetto, ma solo dalla sua presenza e forma. In altre parole, il modello impara a riconoscere un oggetto indipendentemente da dove si trovi nell'immagine.

Inoltre, desideriamo che la funzione di classificazione soddisfi:

$$y(T_v f) = y(f) \quad \forall f, T_v$$

Ovvero, che l'**output del modello sia invariante rispetto alla traslazione** del contenuto. Questa invarianza è **desiderabile anche su scale multiple**, cioè vogliamo che anche patch locali dell'immagine, di dimensione variabile, siano invarianti rispetto alla loro posizione:

$$z(T_v p) = z(p) \quad \forall p, T_v$$

dove p sono porzioni dell'immagine (patch) a diverse scale.

5.4 Convolutional Neural Networks (CNN)

5.4.1 Operazione di Convoluzione

Nel contesto delle CNN, la convoluzione è un'operazione fondamentale.

La convoluzione prende un'immagine (o in generale una funzione) e un piccolo filtro (il kernel), e li fa interagire in modo sistematico: il filtro viene fatto **scorrere su tutta l'immagine**, e in ogni posizione si calcola un prodotto tra il filtro e la porzione dell'immagine sottostante. Questo prodotto viene poi **sommato** e memorizzato come un nuovo valore in una mappa di output (detta mappa delle attivazioni o feature map). Ripetendo questa operazione su tutte le posizioni, otteniamo un'immagine trasformata che mette in evidenza le strutture che assomigliano al filtro usato.

In sintesi, la convoluzione:

- **cattura relazioni locali** tra i dati;
- è traslazionalmente equivariante, cioè reagisce coerentemente agli spostamenti nell'input;
- consente di costruire rappresentazioni più astratte e informative via via che si applicano più convoluzioni.

5.4.2 Proprietà della Convoluzione

1. Commutativa

L'ordine con cui si applicano le funzioni nella convoluzione non cambia il risultato.

2. Shift Equivariance:

Questa proprietà afferma che traslare l'input produce una traslazione identica dell'output, rendendo l'operazione di convoluzione equivariante alla traslazione.

L'equivarianza è una proprietà chiave della convoluzione: il modello "segue" il movimento dell'oggetto nell'immagine.

5.4.3 Interpretazione Classica: Finestra Mobile

La classica interpretazione della convoluzione come una finestra che si muove sull'immagine. In ogni posizione, si confronta il contenuto locale dell'immagine con il filtro, ottenendo un numero che va a costituire l'output (Immagine ad esempio nel Capitolo 5.4.8).

Il **filtro** (o kernel) è un quadrato contenente **i parametri che dobbiamo ottimizzare** durante il training. Ad esempio, nel caso di un filtro 3x3, ci sono 9 parametri da apprendere.

Quando questa finestra si muove sull'immagine, riesce a rilevare la presenza (o assenza) di certi pattern in regioni diverse, fornendo così una rappresentazione più astratta e significativa del contenuto visivo.

5.4.4 Architettura delle CNN

Le CNN sfruttano il fatto che **i dati visivi spesso presentano pattern locali, gerarchici e invarianza rispetto alle traslazioni**. Questo viene assunto **come un prior strutturale** e sfruttato direttamente nell'architettura.

5.4.5 Differenza tra layer fully-connected e layer convoluzionale

- Nei **fully-connected layer**, ogni nodo è connesso a tutti i nodi dello strato precedente, con pesi indipendenti.
- Nei **convolutional layer**, invece, si utilizza il concetto di **condivisione dei pesi** (*weight sharing*): lo stesso filtro viene applicato a tutte le regioni dell'immagine.

Questo riduce enormemente il numero di parametri e sfrutta la struttura locale dei dati.

5.4.6 Idea chiave: strati equivarianti costruiti via convoluzione

1. Singolo strato convoluzionale

Lo strato convoluzionale applica un insieme di pesi ai canali in input:

$$g_{\ell}(x) = \sigma\left(\sum_{\ell'=1}^n (f_{\ell'} \cdot w_{\ell\ell'})(x)\right)$$

dove:

- σ è la funzione di attivazione (es. ReLU);
- $w_{\ell\ell'}$ è il filtro associato alla trasformazione da input a output ;
- $f_{\ell'}$ è il canale in input;
- g_{ℓ} è il canale in output.

2. CNN con L strati convoluzionali

In una rete convoluzionale profonda (deep CNN), si concatenano strati convoluzionali.

Questa struttura permette di costruire rappresentazioni **sempre più astratte** e complesse dei dati visivi, man mano che si sale negli strati.

3. CNN con L strati convoluzionali intervallati da pooling

Spesso, tra uno strato convoluzionale e l'altro, viene inserito un **pooling layer**. Il pooling ha lo scopo di **ridurre la dimensionalità** dei dati (downsampling), migliorando l'efficienza computazionale e rendendo la rete più **robusta alle traslazioni locali**.

Questo porta al concetto di **shift-invariance**, cioè la capacità del modello di riconoscere una caratteristica anche se questa è **spostata leggermente** nella posizione dell'immagine.

Ogni filtro (o kernel) è una piccola matrice di pesi che scorre sull'immagine, producendo una nuova immagine chiamata mappa di attivazione (feature map). Ogni filtro è specializzato per rilevare un particolare tipo di caratteristica (bordi, angoli, texture...).

La **shift invariance** viene ottenuta combinando **convoluzioni condivise** (che garantiscono equivarianza) con **pooling locali**, che rendono l'output insensibile a piccoli spostamenti.

5.4.7 Vantaggi delle Convoluzioni

1. $O(1)$ parametri per filtro:

Ogni filtro ha un **numero fisso di parametri**, che dipende solo dalla dimensione del filtro, **non dalla dimensione dell'immagine**.

Questo è un enorme vantaggio rispetto alle reti tradizionali (MLP), che richiedono un numero di parametri **proporzionale alla dimensione dell'input**.

2. **Weight Sharing** (condivisione dei pesi):

Lo **stesso filtro viene applicato a tutta l'immagine**: non servono parametri distinti per ogni posizione.

Questo riduce drasticamente il numero totale di parametri e rende il modello **più efficiente** e **più robusto** agli spostamenti.

Il weight sharing è il motivo per cui le convoluzioni sono robuste rispetto alle traslazioni all'interno dell'immagine.

5.4.8 Pooling: Funzione e Utilità

Nelle **Convolutional Neural Networks (CNN)**, i **pooling layer** vengono tipicamente inseriti tra strati convoluzionali successivi. La loro funzione principale è:

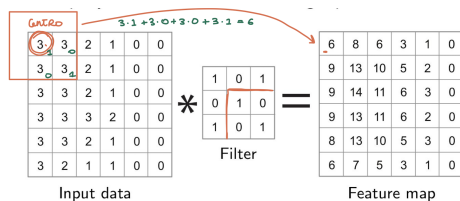
- **Ridurre la dimensionalità** dell'output degli strati convoluzionali;
- **Migliorare la robustezza** del modello rispetto a traslazioni locali;
- **Aggregare informazioni** su regioni più ampie dell'immagine.

Un'operazione di pooling comune è il **max pooling**, che seleziona il valore massimo in una finestra locale (ad esempio 2×2) e lo usa come rappresentazione aggregata di quella regione.

Questa operazione permette al modello di:

- Catturare **interazioni non locali** usando blocchi semplici e sparsi;
- Rendere più efficiente il calcolo e limitare l'overfitting.

Esempio Visivo di Pooling



L'input è una matrice (immagine o feature map)
Un filtro convoluzionale viene applicato su un receptive field (una zona dell'immagine) e produce una nuova mappa (feature map)

Il **max pooling** riduce questa mappa scegliendo il massimo in ogni regione locale. Il risultato è una nuova mappa più piccola ma che conserva le informazioni più significative.

13	10	2
14	11	3
13	10	3

Max pooling

5.4.9 Apprendimento Gerarchico delle Features

Le CNN sono in grado di apprendere **feature a livelli crescenti di astrazione**:

1. Low-Level Features

Bordi, angoli, linee, texture semplici.

Sono le prime feature apprese, presenti nella maggior parte delle immagini.

Ogni filtro apprende un pattern visivo comune e riutilizzabile.

2. Mid-Level Features

Combinazioni più complesse di pattern semplici.

Possono rappresentare parti di oggetti (es. fari, ruote, finestre).

Sono meno generiche e più informative per specifici compiti.

3. High-Level Features:

Rappresentano le caratteristiche più astratte e specifiche.

Possono essere parti significative dell'oggetto target (es. parti di un'auto).

Il modello, in questa fase, riconosce forme o pattern distintivi per la classe (es. "car").

Il **pooling** consente alla rete di semplificare l'informazione mantenendo ciò che è rilevante.

Le CNN **estraggono progressivamente** caratteristiche più complesse man mano che si approfondisce nella rete.

Questo approccio gerarchico è alla base del successo delle CNN nella **visione artificiale**, poiché permette di passare dai pixel grezzi a decisioni ad alto livello come il riconoscimento di oggetti.

5.5 Proprietà Chiave delle CNN

1. Filtri Convoluzionali (Equivarianza alla Traslazione)

Le convoluzioni rilevano feature (bordi, forme) in tutta l'immagine.

L'equivarianza alla traslazione permette di riconoscere un oggetto anche se spostato.

2. Strati Multipli (Composizionalità)

- I primi layer rilevano feature semplici (bordi, angoli).
- I layer successivi combinano queste feature in pattern più complessi.
- Gli ultimi layer apprendono rappresentazioni di alto livello (oggetti interi).

3. Filtri Localizzati nello Spazio (Località)

I filtri osservano solo porzioni locali dell'immagine (*receptive fields*).

La rete può così **catturare dettagli locali** in modo efficace.

4. Condivisione dei Pesi (Self-similarity)

Lo stesso filtro viene applicato ovunque.

Questo riduce enormemente il numero di parametri da allenare.

5. $O(1)$ Parametri per Filtro

I parametri dipendono solo dalla **dimensione del filtro**, non dell'immagine.

Le CNN sono molto più efficienti delle MLP su immagini grandi.

5.5.1 Componenti di una Rete Convoluzionale

1. Input

2. Convolutional Layer

- L'input mantiene la sua struttura spaziale (es. $32 \times 32 \times 3$).
- Il **filtro** (es. $5 \times 5 \times 3$) scorre sull'immagine (*sliding window*).
- In ogni posizione:
 - Si esegue un **prodotto scalare** tra i valori del filtro e quelli dell'immagine coperti.
 - Si ottiene un **singolo numero**, che va nella **feature map**.

3. Funzioni di attivazione

Introdotte dopo ogni layer per introdurre non linearità.

Le più comuni includono: **ReLU**, **sigmoid**, **tanh**.

Permettono al modello di apprendere funzioni complesse.

4. Normalization

Durante l'addestramento, le attivazioni possono cambiare distribuzione (*internal covariate shift*).

Per mitigare questo effetto si usa la **Batch Normalization**, che normalizza le attivazioni di un layer in modo che abbiano media zero e varianza unitaria.

5. Pooling Layer

Riduce la dimensione spaziale dell'input (es. da 32×32 a 16×16).

Tipicamente usa operazioni come **max pooling** o **average pooling**.

Aiuta a rendere il modello più robusto a traslazioni e riduce il numero di parametri.

6. Possibilmente altri blocchi Conv + ReLU + Pooling ...

7. Flatten:

L'input (le feature map, es. $32 \times 32 \times 3$) viene **appiattito** in un vettore ($32 \times 32 \times 3 = 3072$).

8. Fully Connected Layer

Il vettore viene passato a uno o più **fully connected layers** e moltiplicato per una matrice di pesi W (es. 10×3072) $\rightarrow$ una riga per ogni classe.

Ogni prodotto scalare tra riga di W e input produce un'**attivazione**. L'ultimo strato fully connected genera l'**output finale**, ad esempio:

10 neuroni $\rightarrow$ classificazione su 10 classi (con softmax).

Il risultato della convoluzione è una nuova rappresentazione dell'immagine, detta feature map. La dimensione della mappa può essere:

- più piccola se non si usa padding (es. da 32×32 a 28×28)
- uguale se si applica padding appropriato (es. "same" padding)

Una CNN è una rete che unisce convolutional layers (per rappresentazione) e fully connected layers (per decisione).

In modelli moderni (tipo ResNet), a volte si elimina del tutto il fully connected layer (tranne l'ultimo), sostituendolo con un **global average pooling**, per ridurre overfitting e parametri.

5.5.2 Funzioni di Attivazione

Le funzioni di attivazione introducono **non-linearità** nei modelli neurali, permettendo di apprendere funzioni complesse. Vediamo le principali:

1. Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in [0, 1]$$

Problemi:

- **Saturazione** per valori estremi $\rightarrow$ il gradiente tende a zero.
- **Non centrata nello zero** $\rightarrow$ produce gradienti sempre dello stesso segno.
- Usa `exp()`, che è costosa da calcolare.

2. Tanh

$$\tanh(x) \in [-1, 1]$$

- È centrata nello zero.
- **Problema:** satura comunque e riduce i gradienti (vanishing gradient).

3. ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x) \in [0, +\infty)$$

- **Non satura** per $x > 0$.
- Molto **efficiente** computazionalmente.
- Converge più velocemente rispetto a sigmoid/tanh (fino a $6\times$).
- **Problema:** non centrata nello zero e **”muore”** per $x < 0$ (gradiente nullo).
- La metà dei neuroni (quelli che ricevono input negativi) restituisce zero $\rightarrow$ la **varianza del segnale si dimezza circa**.
- Soluzione comune: inizializzazione con **bias positivi piccoli** (es. 0.01).

4. Leaky ReLU

$$\text{LReLU}(x) = \max(0.01x, x)$$

- Come la ReLU, ma con una piccola pendenza per $x < 0$.
- Non “muore” mai.
- Variante: **PReLU**, in cui il coefficiente negativo è appreso dal modello.

5.5.3 Regola generale

1. Usa **ReLU** nella maggior parte dei casi
2. Prova Leaky ReLU / ELU se vuoi migliorare ulteriormente
3. **Evita sigmoid/tanh** nei hidden layer

—

6 Preprocessing dei dati

Nel contesto del machine learning e, in particolare, delle reti neurali, preprocessare i dati è un passo fondamentale per ottenere un apprendimento efficace. Questa lezione approfondisce le trasformazioni applicate ai dati prima dell’addestramento, con particolare attenzione a decorrelazione e whitening.

1. Dati originali

I dati grezzi spesso presentano correlazioni tra le feature. In uno spazio bidimensionale, per esempio, questo si manifesta come una distribuzione ellittica inclinata. In questa configurazione, alcune direzioni nello spazio delle feature mostrano varianza maggiore di altre, e ciò rende l’ottimizzazione dei pesi nella rete neurale più complicata: la loss diventa molto sensibile a piccole variazioni nei pesi.

2. Decorrelazione tramite PCA

Un primo passo utile nel preprocessing è la **decorrelazione**, cioè trasformare i dati in modo che le feature non siano più correlate tra loro. Questo si ottiene applicando la **Principal Component Analysis (PCA)**, che ruota i dati nelle direzioni di massima varianza. Dopo questa rotazione, le componenti principali risultanti sono **linearmente indipendenti** e la matrice di covarianza diventa **diagonale**.

Come funziona la PCA? Si calcola la matrice di covarianza dei dati **centrati**, si eseguono autovalori e autovettori, e si proiettano i dati lungo gli autovettori corrispondenti ai maggiori autovalori. Questi rappresentano le direzioni di massima varianza.

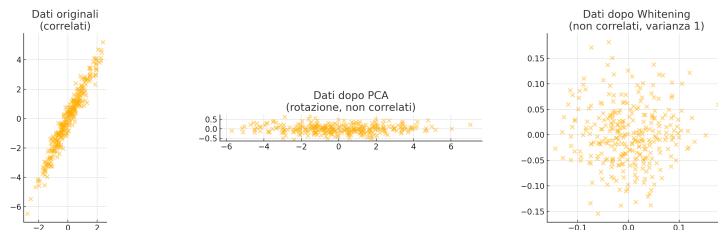
3. Whitening

Il **whitening** è un passo successivo che va oltre la sola decorrelazione. Dopo aver ottenuto i dati decorrelati tramite PCA, si **normalizza la varianza di ciascuna componente**. Questo significa dividere ogni componente per la radice quadrata del proprio autovalore (cioè la varianza in quella direzione). Il risultato finale è che:

- Le feature sono scorrelate;
- Tutte hanno **varianza unitaria**;
- La matrice di covarianza è **l'identità**.

In sintesi: la PCA fornisce decorrelazione (covarianza diagonale), mentre il whitening aggiunge la normalizzazione (covarianza = identità). Il whitening può essere implementato con PCA, ma non si riduce solo alla PCA.

Schema riassuntivo del processo:



6.0.1 Effetti sul training

- **Prima della normalizzazione**, la funzione di perdita per la classificazione è molto sensibile a piccoli cambiamenti nei pesi. In uno spazio distorto, un piccolo aggiornamento dei pesi può produrre cambiamenti enormi in alcune direzioni e nulli in altre.
- **Dopo la normalizzazione**, i dati sono distribuiti in modo più uniforme e isotropo. Questo rende la funzione di perdita più ben comportata e quindi l'ottimizzazione più semplice e stabile. I gradienti diventano meno soggetti a variazioni brusche, migliorando la convergenza dell'algoritmo di apprendimento.

Perché l'ottimizzazione migliora dopo la normalizzazione? Perché si riduce l'anisotropia del problema: tutti i gradienti agiscono su dimensioni con la stessa scala e importanza, semplificando il lavoro dell'ottimizzatore (es. discesa del gradiente).

6.0.2 Normalizzazione nelle immagini

Nel caso specifico delle immagini, ad esempio nel dataset CIFAR-10 ([32, 32, 3]), è comune centrare i dati ma meno comune normalizzarli in varianza. Le tecniche di preprocessing più usate sono:

- **Sottrarre l'immagine media** (es. AlexNet): si calcola la media su tutto il dataset e si sottrae da ogni immagine.
- **Sottrarre la media per canale** (es. VGGNet): si calcola la media lungo ciascun canale colore (RGB) e si sottrae.
- **Sottrarre la media e dividere per la deviazione standard per canale** (es. ResNet): normalizzazione più spinta, utile per rendere le reti più stabili durante l'addestramento.

Tuttavia, si evita spesso la normalizzazione della varianza tramite whitening per le immagini, perché:

- Aumenta la complessità computazionale;
- Le reti convoluzionali sono già robuste a certe distorsioni nei dati grezzi.

6.1 Inizializzazione dei pesi e symmetry breaking

6.1.1 Cos'è il symmetry breaking

Quando inizializzi tutti i pesi $W = 0$ e i bias $b = 0$, ogni neurone nella rete riceve esattamente lo stesso input, produce lo stesso output e riceve lo stesso gradiente. Quindi tutti i neuroni restano identici anche dopo l'aggiornamento dei pesi.

Tutti i neuroni si aggiornano allo stesso modo → **non si differenziano mai** → stanno imparando esattamente la **stessa cosa** → la rete non può apprendere funzioni complesse.

Non c'è **rottura di simmetria** (*symmetry breaking*), ovvero non c'è diversificazione tra le unità della rete. Questo impedisce alla rete di apprendere funzioni complesse.

6.1.2 Soluzioni proposte

1. Aumentare la deviazione standard dei pesi iniziali

Inizializzare i pesi con valori casuali da una distribuzione gaussiana a media zero e **aumentare la deviazione standard** da 0.01 a 0.05.

- Aiuta a evitare che i segnali si spengano troppo in fretta attraversando i layer.
- Ma se troppo grande, porta a **saturazione** (es. $\tanh$) e scomparsa dei gradienti.

2. Xavier Initialization

Una soluzione sistematica è l'inizializzazione Xavier:

$$W = \frac{\text{randn}(D_{in}, D_{out})}{\sqrt{D_{in}}}$$

Ossia si genera una matrice di numeri random da una distribuzione gaussiana $N(0, 1)$ e si normalizza la varianza.

- Mantiene costante la varianza tra input e output.
- Funziona bene con attivazioni **simmetriche e centrate** (es. $\tanh$).

3. Passare da $\tanh$ a ReLU

ReLU non satura facilmente, ma:

- **Non è centrata in zero.**
- Xavier non è più adatta → le attivazioni possono collassare a zero → bisogna **cambiare inizializzazione**.

4. ReLU correction (He/MSRA Initialization)

Si corregge impostando:

$$W = \text{randn}(D_{in}, D_{out}) \cdot \sqrt{\frac{2}{D_{in}}}$$

Questa è la **He initialization** (o MSRA), che tiene conto della perdita di varianza introdotta da ReLU (che annulla circa metà degli input).

6.1.3 ResNet - Reti Residuali

Nelle reti neurali profonde, uno dei problemi principali è che man mano che si aggiungono strati, diventa difficile mantenere stabile la propagazione delle informazioni. Le **ResNet** (Residual Networks) affrontano questo problema introducendo una struttura che somma l'input originale con il risultato di una trasformazione appresa dal modello. In altre parole, l'output di un blocco residuo è:

$$y = x + F(x)$$

dove $F(x)$ è il risultato della trasformazione appresa all'interno del blocco (tipicamente una sequenza di convoluzioni e ReLU).

Il problema è che, se inizializziamo $F(x)$ usando MSRA (che imposta $\text{var}(F(x)) = \text{var}(x)$), quando sommiamo $x + F(x)$ otteniamo: $\text{var}(y) = \text{var}(x + F(x)) > \text{var}(x)$

Questa crescita della varianza si accumula a ogni blocco della rete, rendendo l'allenamento instabile nelle reti molto profonde.

Per evitare questo problema, si utilizza una strategia semplice ma efficace:

- si inizializza la **prima convoluzione** del blocco con MSRA, come previsto per ReLU;
- ma si inizializza la **seconda convoluzione** del blocco a zero.

In questo modo, inizialmente il valore di $F(x)$ sarà prossimo a zero, e quindi anche $y = x + F(x)$ sarà inizialmente quasi uguale a x . Questo permette alla rete di partire da una configurazione stabile, in cui la varianza non cresce artificialmente tra i blocchi.

Man mano che l'allenamento prosegue, $F(x)$ inizia ad apprendere, e la rete può beneficiare della flessibilità dei blocchi residui mantenendo la stabilità iniziale.

6.2 Regularization

La regularization è una tecnica fondamentale per migliorare le prestazioni di un modello e prevenire l'overfitting. Consiste nell'aggiungere un termine al valore della funzione di loss che penalizza la complessità del modello. Questo termine di penalizzazione dipende dai pesi del modello e viene moltiplicato per un coefficiente lambda, che controlla quanto fortemente penalizzare.

Le forme principali di regularization sono:

- **L2 regularization** (nota anche come weight decay): penalizza la somma dei quadrati dei pesi. Spinge i pesi a essere piccoli ma non necessariamente nulli.
- **L1 regularization**: penalizza la somma dei valori assoluti dei pesi. Ha l'effetto collaterale di spingere alcuni pesi esattamente a zero, quindi promuove la sparsità.
- **Elastic net**: combina L1 e L2 per ottenere sia controllo della magnitudine che sparsità.

6.2.1 Dropout

Una forma particolare di regularization molto usata nelle reti neurali è il **dropout**.

Durante ogni forward pass dell'allenamento:

1. Per ogni neurone in uno strato (tipicamente nei fully connected layer),
2. Si estrae un numero casuale da una distribuzione uniforme tra 0 e 1,
3. Se quel numero è maggiore della dropout rate (es. 0.5), **il neurone viene disattivato** (cioè il suo output è messo a zero).

Questa probabilità viene chiamata **dropout rate**:

La dropout rate tipicamente è 0.5 $\rightarrow$ ogni neurone ha il 50% di possibilità di essere spento in quella iterazione.

La scelta è **diversa ad ogni forward pass, quindi i neuroni disattivati cambiano di continuo durante l'addestramento.**

Questa tecnica costringe la rete a non dipendere troppo da specifici neuroni, inducendo una rappresentazione più ridondante e robusta. In altre parole, la rete impara a distribuire l'informazione in modo più uniforme.

Un'altra interpretazione è che il dropout equivale a **allenare una media di tantissimi modelli diversi** che condividono i pesi: ogni combinazione di neuroni attivi è come un sottomodello.

Durante il test, però, non possiamo usare dropout (perché non vogliamo casualità): allora usiamo tutti i neuroni ma **ridimensioniamo le attivazioni** moltiplicandole per la probabilità di mantenimento ($1 - \text{dropout rate}$), in modo da compensare per il fatto che durante l'addestramento i neuroni erano parzialmente spenti.

Per esempio, se durante l'addestramento un neurone era attivo con probabilità 0.5, al test-time il suo output viene moltiplicato per 0.5.

In pratica:

- Durante l'addestramento: si applica dropout (spegnimento casuale di neuroni);
- Durante il test: si usano tutti i neuroni ma si scalano le attivazioni.

Il dropout è stato largamente usato in architetture come **AlexNet** e **VGG**, specialmente nei fully-connected layer, dove il numero di parametri è molto alto. Architetture successive come **GoogLeNet**, **ResNet** e altre non usano dropout: preferiscono strategie alternative come il global average pooling per evitare overfitting.

6.2.2 Pattern comune nella regolarizzazione

Introdurre casualità nel training, rimuoverla nel test

Molte tecniche di regolarizzazione, anche se diverse tra loro, si basano su un principio comune: **durante il training introducono una componente di casualità**, mentre **nel test-time la rete si comporta in modo deterministico, come se stesse facendo una media** dei comportamenti osservati durante l'allenamento.

6.2.3 Durante il training

La rete viene "disturbata" artificialmente, introducendo variazioni casuali in modo da:

- non dipendere troppo da singoli neuroni o caratteristiche specifiche;
- diventare più robusta a piccoli cambiamenti;

- evitare l'overfitting.

Questi disturbi possono essere:

- spegnere neuroni (Dropout);
- cambiare l'input (Data Augmentation);
- normalizzare su batch diversi (Batch Normalization);
- spegnere layer interi (Stochastic Depth);
- mescolare immagini (Mixup).

6.2.4 Durante il test

La rete **non applica più la casualità**. Invece, assume un comportamento **che rappresenta l'effetto medio** di tutti quei disturbi introdotti nel training. Non c'è più aleatorietà, ma una risposta stabile e coerente.

Questo approccio si può rappresentare con la formula:

"L'output del modello è la media dell'output visto durante il training sotto perturbazioni casuali."

6.2.5 Tecniche che seguono questo pattern

1. **Dropout** – spegne neuroni casuali durante il training, ma nel test li usa tutti e scala l'attivazione.
2. **Batch Normalization** – normalizza su statistiche del batch durante il training; usa statistiche fisse nel test.
3. **Data Augmentation** – trasforma gli input (rotazioni, rumore) durante il training, non nel test.
4. **DropConnect** – disconnette pesi a caso invece che neuroni.
5. **Fractional Max Pooling** – cambia casualmente la posizione del pooling.
6. **Stochastic Depth** – spegne interi blocchi nelle reti profonde, poi li riattiva nel test.
7. **Cutout / Random Erasing** – nasconde a caso porzioni dell'immagine.
8. **Mixup** – combina immagini e label in modo casuale durante il training.

6.2.6 Alcune buone pratiche

- Considera l'uso del **dropout nei fully-connected layer**, specialmente in architetture con molti parametri (es. VGG, AlexNet).
- **Batch normalization** e **data augmentation** sono utili quasi sempre.
- **Cutout** e **Mixup** sono particolarmente efficaci su dataset piccoli e rumorosi.

6.2.7 Batch Normalization (esempio chiave del pattern)

- **Durante il training:** normalizza i dati usando media e deviazione calcolate su ogni mini-batch.
- **Durante il test:** usa media e deviazione pre-calcolate sull'intero dataset per avere stabilità.

6.3 Data Augmentation

La **Data augmentation** è una tecnica che serve ad aumentare artificialmente la dimensione di un dataset creando versioni modificate dei dati esistenti. Lo scopo è migliorare la capacità di generalizzazione del modello, rendendolo più robusto rispetto alle variazioni presenti nei dati reali e meno incline all'overfitting.

Le trasformazioni vengono applicate direttamente ai dati di input (ad esempio immagini), **senza cambiare l'etichetta associata**. Questo permette al modello di imparare a riconoscere gli stessi oggetti anche se si presentano in condizioni leggermente diverse.

Trasformazioni principali:

- **Flip orizzontale**

Capovolgere l'immagine lungo l'asse verticale. È utile quando la classe dell'immagine non dipende dalla direzione (es. gatti, volti, oggetti simmetrici).

- **Random crops e scaling**

Usato ad esempio in ResNet:

- Durante il **training**:

1. Si sceglie una dimensione casuale L tra 256 e 480;
2. Si ridimensiona l'immagine affinché il lato corto sia L ;
3. Si estrae a caso una patch 224×224 dall'immagine.

- Durante il **test**:

1. Si ridimensiona l'immagine in 5 scale fisse: 224, 256, 384, 480, 640;
2. Per ogni scala si estraggono 10 patch 224×224 (i 4 angoli, il centro, e i rispettivi flip);
3. Il risultato finale è dato dalla media delle predizioni.

- **Color jitter**

Consiste nel variare i colori in modo controllato, in due varianti:

- **Semplice**: randomizzare luminosità, contrasto, saturazione.
- **Complesso**: applicare PCA ai pixel (sui canali R, G, B), campionare una direzione nello spazio delle componenti principali, e sommare uno *spostamento di colore* (color offset) a tutti i pixel dell'immagine lungo quella direzione.

- **Altre trasformazioni possibili**

Si possono combinare in modo casuale altre trasformazioni, come:

- Traslazioni,
- Rotazioni,
- Zoom e scaling non proporzionale (stretching),
- Distorsioni ottiche (lens distortion),
- Shearing (taglio angolare),
- Altri effetti specifici dell'ambiente reale (es. rumore, sfocature).

Il vantaggio è che **ogni trasformazione contribuisce ad aumentare la varietà del training set**, simulando condizioni reali più varie senza bisogno di raccogliere nuovi dati.

6.4 Training Dynamics

Durante l'addestramento di una rete neurale, oltre alla struttura iniziale (attivazioni, normalizzazione, inizializzazione, regolarizzazione), è fondamentale **monitorare e controllare dinamicamente** l'andamento del processo di apprendimento. Questo include:

1. **One-time setup** – Attivazioni, preprocessing, inizializzazione dei pesi, regolarizzazione.
2. **Training dynamics** – Supervisione del processo di apprendimento, aggiornamento dei parametri, tuning degli iperparametri.
3. **Post-training** – Uso di ensemble di modelli, trasferimento di conoscenza (transfer learning).

6.4.1 Babysitting the learning process

Il processo di ottimizzazione (tipicamente discesa del gradiente) può essere visto come un cammino in un paesaggio di perdita: a ogni passo, si sceglie la direzione in cui la funzione di loss

diminuisce più velocemente (cioè il gradiente).

Questo richiede una supervisione attenta durante il training: la rete può convergere troppo lentamente, oppure oscillare e non stabilizzarsi. Un parametro cruciale per controllare questa dinamica è il **learning rate**.

6.4.2 Learning Rate

Tutti gli ottimizzatori comuni (SGD, SGD con momentum, RMSProp, Adam, ecc.) usano il **learning rate** come iperparametro: esso determina l'ampiezza dei passi compiuti nella discesa del gradiente.

Un **learning rate troppo alto** può causare divergenza (il modello non converge), mentre uno **troppo basso** rallenta l'apprendimento. La pratica comune è iniziare con un learning rate alto e **decrederlo gradualmente** nel tempo.

6.4.3 Learning Rate Decay

Ridurre il learning rate nel tempo migliora la stabilità e la precisione finale del modello. Inizialmente si fanno passi più grandi per convergere velocemente, poi più piccoli per affinare la soluzione e non "saltare" oltre il minimo. Le strategie più comuni sono:

- **Step decay**: riduci il learning rate in punti prefissati (es. per ResNet: dopo 30, 60, 90 epoche, moltiplicalo per 0.1).
- **Cosine annealing**: decresce in modo sinusoidale verso zero.
- **Linear decay**: decresce linearmente con il tempo.
- **Inverse square root**: decresce velocemente all'inizio, poi più lentamente.
- **Constant**: mantiene il learning rate fisso per tutta la durata.

La scelta della strategia può avere un impatto rilevante sull'efficacia dell'ottimizzazione.

6.4.4 Early Stopping

Un'altra tecnica utile è l'**early stopping**. Consiste nel **interrompere l'allenamento quando le prestazioni sul validation set iniziano a peggiorare**.

Questo aiuta a evitare l'overfitting e a risparmiare tempo computazionale. In alternativa, si può continuare a lungo l'allenamento ma **salvare lo snapshot del modello** che ha ottenuto il miglior risultato sulla validation.

In entrambi i casi, l'obiettivo è ottenere un modello che **generalizza bene** su dati non visti.

6.5 Hyperparameter Optimization

Ottimizzare gli iperparametri è un processo cruciale per ottenere le migliori prestazioni da una rete neurale. Gli iperparametri includono il learning rate, il weight decay, l'architettura del modello, i parametri di regolarizzazione, ecc.

6.5.1 Tecniche principali

1. **Grid Search**: si definisce una griglia di possibili valori per ciascun iperparametro e si valutano tutte le combinazioni possibili. Di solito si usano spazi logaritmici (es. learning rate = 0.1, 0.01, 0.001, ...).
2. **Random Search**: si estraggono casualmente combinazioni di valori dagli intervalli possibili. In pratica è spesso più efficace della grid search, perché esplora più direzioni nello spazio degli iperparametri.

6.5.2 Procedura generale

1. **Sanity check iniziale:** disattiva la regolarizzazione e verifica che la loss iniziale abbia senso. Ad esempio, per una softmax su C classi, la loss dovrebbe essere circa $\log(C)$. Se non è così, controlla inizializzazione e attivazioni.
2. **Overfitting su un piccolo sottoinsieme:** prova ad allenare il modello su un sottoinsieme molto piccolo (es. 5–10 mini-batch) e cerca di raggiungere il 100% di accuratezza. Se non ci riesci:
 - la learning rate potrebbe essere troppo bassa,
 - l'inizializzazione potrebbe essere problematica,
 - disattiva la regolarizzazione per essere sicuro che non stia interferendo.
3. **Cerca una learning rate efficace:** usa l'architettura trovata al punto 2 su tutto il dataset, attiva un piccolo weight decay, e cerca una learning rate che faccia scendere chiaramente la loss entro 100 iterazioni. Prova valori come: 10^{-1} , 10^{-2} , 10^{-3} , 10^{-4} .
4. **Training grossolano (coarse grid):** scegli alcune combinazioni di learning rate e weight decay che sembrano promettenti, e allena pochi modelli per 1–5 epoche.
5. **Training raffinato:** scegli i modelli migliori dal punto 4 e allenali più a lungo (10–20 epoche), questa volta senza applicare decay alla learning rate.
6. **Analisi delle learning curve:** la loss può essere rumorosa, quindi osserva le curve con grafici scatter e usa medie mobili per evidenziare le tendenze. Impara a leggere i segnali:
 - se la loss resta alta → learning rate troppo bassa;
 - se esplode → learning rate troppo alta;
 - se c'è plateau → prova a ridurre il learning rate (decay);
 - se train e validation divergono → overfitting.
7. **Iterazione:** torna al punto 5 con le nuove informazioni, e raffina ulteriormente.

6.5.3 Principali iperparametri su cui lavorare

- Architettura della rete (numero e dimensione dei layer);
- Learning rate (e relativo schema di decay);
- Tipologia di aggiornamento (SGD, Adam, etc.);
- Parametri di regolarizzazione (L2, dropout).

6.6 Monitorare l'ampiezza degli aggiornamenti dei pesi

Quando si usa l'SGD, è importante assicurarsi che gli aggiornamenti dei pesi non siano troppo grandi (instabilità) né troppo piccoli (apprendimento lento).

Per farlo, si può tracciare il rapporto:

`ratio = update_scale / param_scale`

Dove:

- `param_scale` è la norma (modulo) dei pesi attuali;
- `update_scale` è la norma del vettore di aggiornamento, cioè $\text{learning_rate} \cdot \nabla$ (gradiente).

6.6.1 Interpretazione del rapporto

- Se **troppo grande** (es. > 0.01): gli aggiornamenti sono troppo forti, il modello può divergere.
- Se **troppo piccolo** (es. < 0.0001): il modello impara troppo lentamente.

L'ideale è mantenere questo rapporto intorno a $1e-3$, in modo che gli aggiornamenti siano significativi ma non destabilizzanti.

7 After Training: Model Ensembles e Transfer Learning

7.0.1 Contesto generale

Dopo aver definito l'architettura della rete (attivazioni, inizializzazione, preprocessing, regolarizzazione) e seguito l'addestramento con attenzione (monitorando gradienti, aggiornamenti, iperparametri, learning rate), esistono tecniche da applicare **dopo il training** che possono migliorare le prestazioni del modello senza modificare la rete o i dati.

Due tecniche principali sono:

- **Model ensembles**
- **Transfer learning**

7.0.2 Model Ensembles

Un **ensemble di modelli** migliora le performance combinando le predizioni di **più modelli indipendenti**. Durante il test, le predizioni di ciascun modello vengono mediate (tipicamente come media aritmetica o voto maggioritario), ottenendo un boost di accuratezza (tipicamente circa 2%).

Perché funziona? Ogni modello ha una propria traiettoria di apprendimento e commette errori diversi. Combinando i loro output, si annulla parte della varianza e si riduce il rischio di overfitting a specifici pattern del training set.

Approccio standard: multiple models Si allenano più modelli separati con la stessa architettura ma inizializzazioni diverse (o con differenti ordini nei dati di training), e si aggregano i loro output nel test.

Approccio efficiente: Snapshot Ensembles Invece di allenare modelli distinti, si salva lo **stesso modello in diversi momenti dell'allenamento** (tipicamente usando un **learning rate ciclico** per forzare il modello a visitare più minimi locali diversi). Questi snapshot vengono poi combinati come se fossero modelli distinti.

- Vantaggio: si ottiene un ensemble **senza costo aggiuntivo significativo** in termini di tempo di training.

Polyak Averaging Un'altra alternativa efficiente è il **Polyak averaging** (o **exponential moving average** dei pesi): durante il training si mantiene una media mobile dei parametri del modello. Al test-time si usano questi pesi medi per fare inferenza.

- Effetto: output più stabili e predizioni più robuste.
- Computazionalmente leggero: aggiornare una media mobile è poco costoso.

Gli ensemble – sotto forma di modelli distinti, snapshot o media dei pesi – sono strumenti molto potenti per migliorare le prestazioni finali del modello con **basso costo computazionale aggiuntivo**.

La scelta tra le opzioni dipende dal budget disponibile e dal tipo di rete o compito, ma snapshot ensemble e Polyak averaging sono strategie particolarmente eleganti per ottenere benefici significativi in fase di test.

7.0.3 Transfer Learning

Il **transfer learning** consiste nel riutilizzare un modello addestrato su un compito (source) per un altro compito diverso ma correlato (target). Per esempio, i modelli pre-addestrati su **ImageNet** (dataset con oltre 14 milioni di immagini annotate in 1000 categorie) vengono spesso usati come punto di partenza per altri compiti di visione artificiale come classificazione o rilevamento di oggetti.

Grazie al knowledge acquisito su ImageNet (che cattura caratteristiche visive generali), si può:

- Ridurre i tempi di training
- Lavorare anche con dataset target più piccoli
- Evitare l'overfitting

Questi sono i principali vantaggi del **transfer learning**, una tecnica che consiste nel riutilizzare una rete neurale pre-addestrata su un grande dataset per un nuovo compito.

Ecco come si articola il processo:

1. Pre-training su un grande dataset

Una CNN viene inizialmente addestrata su un dataset di grandi dimensioni e molto vario, come ImageNet. La rete contiene una sequenza di strati convoluzionali e di pooling (Conv + MaxPool), seguita da uno o più strati fully connected. L'ultimo layer (ad esempio un FC-1000) produce una predizione per ciascuna delle 1000 classi di ImageNet.

Questo training permette alla rete di apprendere **feature generiche delle immagini**, come bordi, texture, forme e strutture comuni.

2. Riutilizzo su un piccolo dataset (C classi)

Quando si ha a disposizione un dataset target di dimensioni ridotte e con un numero limitato di classi (C), si procede in genere nel seguente modo:

- Si **congela la parte convoluzionale** della rete (Conv + MaxPool), che contiene feature generiche utili a molti compiti visivi;
- Si **rimuove l'ultimo layer** fully connected (ad esempio FC-1000) e lo si sostituisce con un nuovo layer adatto al numero di classi C ;
- Si **addestra solo questo ultimo layer** sul nuovo dataset.

Questo approccio è efficiente, previene l'overfitting e sfrutta i pesi pre-addestrati già ottimizzati.

3. Fine-tuning su un dataset più grande

Se il dataset target è sufficientemente ampio, si può ottenere un miglioramento ulteriore tramite **fine-tuning**. In questo caso:

- Si riaddestrano anche alcuni layer intermedi della rete;
- I layer iniziali, che apprendono feature molto generali, vengono solitamente **congelati**;
- I layer più vicini all'output vengono **aggiornati**, perché sono quelli più specializzabili;
- Si utilizza un **learning rate più basso** (circa 1/10 di quello iniziale) per evitare di cancellare l'informazione appresa nel pre-training.

7.0.4 Considerazioni finali

- I layer iniziali imparano feature molto generiche (es. bordi), i layer profondi caratteristiche più specifiche.
- L'efficacia del transfer learning dipende da:
 1. **Somiglianza del dataset** (source vs target);
 2. **Quantità di dati disponibili**.

7.0.5 Linee guida

Durante il fine-tuning in un contesto di transfer learning, si aggiornano i pesi dei layer selezionati permettendo loro di essere modificati durante l'addestramento. Questo processo consente al modello di adattarsi meglio al nuovo compito, specialmente quando il dataset target è sufficientemente grande o simile a quello originale.

Dati disponibili / Similarità	Dataset simile	Dataset diverso
Pochi dati (10–100/class)	Usa un classificatore lineare	Situazione critica; prova varie combinazioni
Molti dati (100–1000/class)	Fine-tuning di pochi layer	Fine-tuning di più layer

Table 1: Strategie di trasferimento in base alla quantità di dati e alla similarità del dataset

Aggiornare troppi layer con un dataset piccolo può portare a overfitting. È consigliabile limitare l'aggiornamento ai layer finali in questi casi. Durante il fine-tuning, è importante gestire correttamente i layer di Batch Normalization, mantenendoli in modalità inferenza per evitare comportamenti indesiderati.

I miglioramenti nelle architetture CNN moderne si riflettono direttamente in migliori risultati nei task downstream, grazie alla maggiore efficacia del transfer learning.

Applicazioni avanzate di Transfer Learning

Il transfer learning è molto versatile e viene ormai applicato anche in ambiti multimodali. Esempio pratico:

1. Si allena una CNN su ImageNet.
2. Si fa **fine-tuning** di questo modello per un compito specifico, ad esempio *object detection*.
3. In parallelo, si allena un modello linguistico (es. *BERT*) su grandi corpus di testo.
4. Si combinano i due modelli per affrontare compiti che richiedono sia visione che linguaggio, come *image captioning* o *visual question answering*.
5. Infine, si fa **fine-tuning** sul compito target multimodale.

Alcuni studi recenti mostrano che:

- Addestrare da zero (training from scratch) può funzionare **altrettanto bene**, se si allena per molto più tempo e con più dati.
- Tuttavia, **pretraining + fine-tuning** è ancora superiore **quando il dataset target è piccolo**.
- Aumentare la quantità di dati è spesso più efficace che preaddestrare.

Considerazioni finali:

- Pretraining + fine-tuning rimane una strategia potentissima per velocizzare il training e ottenere ottime performance anche con pochi dati.
- L'addestramento da zero è competitivo solo in scenari con grandi dataset e risorse computazionali.
- La ricerca su come e quando usare il transfer learning è ancora in evoluzione: molte pratiche vanno affinate.

7.1 Architetture di Reti Convoluzionali (CNN)

7.1.1 AlexNet

AlexNet è una delle prime reti CNN profonde ad aver rivoluzionato la visione artificiale. La rete ha 5 strati convoluzionali seguiti da 3 fully-connected (FC).

- **Conv layers:** usano kernel grandi inizialmente (es. 11x11 in conv1), stride 1 e padding. Man mano che si va avanti, la dimensione del kernel si riduce.
- **Pool layers:** intervallati tra i conv, riducono la dimensionalità.
- **FC layers:** sono la parte più pesante in termini di parametri.

Costi computazionali

- La memoria viene usata soprattutto nei primi layer convoluzionali.
- Quasi tutti i parametri sono nei fully-connected layers.
- La maggior parte delle operazioni floating-point (FLOP) avviene nei conv layers.

7.1.2 VGG

VGG (VGG16/VGG19) semplifica la struttura usando solo convoluzioni 3x3 (stride 1, pad 1) e pooling 2x2 (stride 2):

- Tutti i layer convoluzionali hanno receptive field equivalenti ottenuti tramite più 3x3.
- Dopo ogni max-pooling, si raddoppiano i canali.
- La rete ha 5 stadi: ogni stadio è una combinazione di conv-conv-pool, con profondità crescente.

VGG16 è molto più grande di AlexNet:

- 25x più memoria,
- 2.3x più parametri,
- 19x più FLOP.

7.1.3 GoogLeNet (Inception)

GoogLeNet introduce il modulo **Inception**, che migliora l'efficienza e riduce la complessità.

- Nessun fully-connected layer alla fine.
- Solo 5 milioni di parametri.
- Architettura vincente di ILSVRC 2014.

Componenti principali:

1. **Stem network:** downsampling aggressivo per ridurre la risoluzione iniziale.
2. **Inception module:** blocchi paralleli con convoluzioni 1×1 , 3×3 , 5×5 che riducono i canali prima di operazioni costose.
3. **Global Average Pooling:** al posto dei fully connected.
4. **Auxiliary classifiers:** classificatori intermedi per aiutare la propagazione del gradiente.

7.1.4 ResNet

Con Batch Normalization, si possono allenare reti molto più profonde (10+ layer). Tuttavia, senza accorgimenti, i modelli profondi:

- hanno training error più alto,
- tendono all'**underfitting** (!).

Soluzione: Residual Learning Invece di imparare direttamente una funzione, si impara la **residual** tra input e output. Si introduce un collegamento diretto (**skip connection**) tra input e output del blocco.

- Un blocco residuo ha 2 convoluzioni 3x3 + shortcut.
- Ogni stadio dimezza la risoluzione e raddoppia i canali.
- La rete termina con global average pooling.

Varianti:

- **ResNet-18**: 18 layer, 1.8 GFLOP
- **ResNet-34**: 34 layer, 3.6 GFLOP
- **ResNet-50**: 50 layer, usa **bottleneck blocks** con 1x1 - 3x3 - 1x1 convoluzioni
- **ResNet-101/152**: ancora più profonde, ma computazionalmente più pesanti

7.1.5 Confronto complessivo

- **AlexNet**: semplice, pesante in memoria, meno preciso.
- **VGG**: altissima memoria e operazioni, ma ottima accuratezza.
- **GoogLeNet**: molto efficiente.
- **ResNet**: miglior tradeoff tra accuratezza e complessità.

Riflessioni finali

- Inception-v4 combina Inception e ResNet: ottimo compromesso.
- L'evoluzione delle CNN ha mostrato che reti più profonde tendono a performare meglio, ma a costo di maggiore complessità.

Quale architettura usare?

- **Accuratezza prioritaria**: ResNet-50 o ResNet-101.
- **Efficienza per dispositivi mobili o real-time**: MobileNet, ShuffleNet.

Non reinventare la ruota! Usa architetture ben testate.

Riepilogo evolutivo

1. **AlexNet** → **ZFNet** → **VGG**: reti sempre più grandi.
2. **GoogLeNet**: efficienza architetturale.
3. **ResNet**: reti molto profonde grazie al residual learning.
4. **Post-ResNet**: efficienza e mobilità (es. MobileNet).
5. **NAS (Neural Architecture Search)**: automatizzazione della progettazione.

—

8 Sequential Data e Recurrent Neural Network

I dati sequenziali sono una tipologia di dati in cui l'ordine degli elementi ha un ruolo fondamentale. Non si tratta semplicemente di insiemi di valori indipendenti, ma di informazioni in cui la posizione temporale o logica di ciascun elemento influenza la comprensione complessiva. Questo accade, ad esempio, nei video, dove i frame successivi costruiscono una narrazione, oppure nelle serie temporali, come quelle relative all'andamento della borsa, a misurazioni biologiche, climatiche o economiche. Anche i segnali audio, la musica e il linguaggio parlato sono esempi in cui il tempo gioca un ruolo strutturante. Infine, anche il comportamento degli utenti nei siti web può essere rappresentato come una sequenza di azioni che hanno una dipendenza causale o temporale.

Molte applicazioni moderne si basano sull'elaborazione di dati sequenziali. Tra queste si possono citare la traduzione automatica, la generazione automatica di didascalie per immagini, i sistemi di risposta alle domande, la generazione di video, la sintesi vocale e il riconoscimento del parlato.

Per affrontare questi compiti, si utilizzano

1. sia reti neurali “**vanilla**” (cioè standard, senza memoria),
2. sia reti ricorrenti, che invece sono progettate per gestire la dipendenza tra elementi nella sequenza.

Le reti neurali possono essere organizzate in vari schemi di input-output:

- un input e un output singolo (ad esempio, nella classificazione di immagini),
- un input singolo che genera una sequenza (come nella didascalia di un'immagine),
- una sequenza di input che porta a un solo output (come nell'analisi del sentimento di una frase),
- oppure trasformazioni tra sequenze (come nella traduzione automatica o nella classificazione video frame per frame, dove a ogni frame è associata un'etichetta).

Le RNN, sono progettate per mantenere uno **stato interno** che viene aggiornato man mano che si elabora la sequenza. Questo stato funge da memoria temporanea e consente al modello di tenere conto degli elementi precedenti. A ogni passo temporale, il modello aggiorna lo stato ricorrente applicando la stessa funzione e gli stessi parametri. La formula generale di aggiornamento dello stato è:

$$h_t = f_W(h_{t-1}, x_t)$$

dove h_{t-1} è lo stato precedente, x_t è l'input al tempo t , e f_W è una funzione parametrizzata da W , che viene riutilizzata a ogni istante. Il vettore h_t , detto stato nascosto, può essere calcolato esplicitamente come:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t)$$

e l'output del modello in quel momento è:

$$y_t = W_{hy}h_t$$

La funzione di attivazione $\tanh$ viene comunemente usata per comprimere i valori dello stato in un intervallo compreso tra -1 e 1 , rendendo il sistema più stabile e interpretabile.

8.1 RNN Computational Graph

Nel grafo computazionale di una RNN, la struttura si ripete identica per ogni passo temporale, utilizzando gli stessi pesi. Ogni input della sequenza viene processato con la stessa funzione, e lo stato nascosto viene aggiornato e trasmesso al passo successivo, producendo un output che può a sua volta essere utilizzato per il calcolo della perdita (loss).

- Nel caso many-to-many, ad esempio nella traduzione automatica o nella classificazione di sequenze, ogni elemento della sequenza in input produce un output distinto, e la perdita viene calcolata su ogni output.
- In un modello many-to-one, invece, l'intera sequenza in input produce un singolo output finale, come accade nell'analisi del sentimento.
- Al contrario, nel one-to-many, un singolo input può generare una sequenza di output, come avviene nella didascalia di immagini.
- Esiste anche una combinazione di many-to-one seguito da one-to-many, che corrisponde al meccanismo di codifica-decodifica utilizzato in architetture come Seq2Seq.

Un esempio concreto è il language model a livello di carattere. Se prendiamo la parola "hello", ogni carattere viene rappresentato come un vettore one-hot e trasformato attraverso il modello in una previsione per il carattere successivo. L'output atteso è quindi la sequenza spostata di una posizione. Durante l'addestramento, il modello impara a mappare ogni carattere nel suo successivo nel contesto della sequenza.

In fase di test, il modello genera un carattere alla volta e lo reinserisce in input, aggiornando lo stato ricorrente a ogni passo. Inizialmente gli input vengono codificati come vettori one-hot, ma è comune proiettare questi vettori in uno spazio continuo più compatto tramite uno strato di embedding, che consente di apprendere rappresentazioni distribuite più informative.

Il prodotto tra il vettore one-hot e la matrice dei pesi permette, in pratica, di selezionare una colonna della matrice, che corrisponde all'embedding del carattere. Questa operazione viene spesso isolata in uno strato dedicato, chiamato appunto embedding layer.

8.2 Backpropagation Through Time (BPTT)

Il training di una rete ricorrente utilizza una tecnica chiamata **backpropagation through time**. La rete processa l'intera sequenza in avanti per calcolare la perdita, poi ripercorre la sequenza all'indietro per calcolare i gradienti e aggiornare i pesi.

8.2.1 Truncated Backpropagation Through Time

Nel caso di sequenze molto lunghe, si utilizza una variante detta **truncated backpropagation through time**, dove si divide la sequenza in segmenti più brevi. Si esegue la propagazione in avanti lungo tutta la sequenza, mantenendo gli stati nascosti, ma si retropropaga il gradiente solo attraverso un numero limitato di passi. Questo riduce il costo computazionale e aiuta ad affrontare problemi come l'esplosione o la scomparsa del gradiente.

Queste tecniche permettono alle reti ricorrenti di apprendere da sequenze molto lunghe senza perdere completamente l'informazione temporale, e sono fondamentali per l'addestramento efficace di modelli RNN su testi, audio, video e altre forme di dati temporali.

8.3 Generazione di testo e apprendimento progressivo

Durante l'addestramento, possiamo chiedere alla rete di generare nuovo testo. All'inizio l'output sarà completamente casuale, ma man mano che il modello impara a catturare la struttura del linguaggio, il testo prodotto inizia ad assumere una forma riconoscibile. È possibile, ad esempio, addestrare una rete su opere letterarie e osservare come il testo generato inizialmente sia privo di senso, ma gradualmente si trasformi in frasi che imitano lo stile dell'autore originale.

Possiamo addestrare una RNN sui sonetti di Shakespeare. Inizialmente la rete produce solo sequenze casuali di caratteri, ma più viene addestrata e più riesce a replicare lo stile e la sintassi delle poesie.

Questa stessa idea può essere estesa a testi tecnici, come il codice. Una RNN può essere addestrata a produrre codice simile a quello del kernel di Linux. In questo caso, il modello non solo genera caratteri coerenti con il lessico del linguaggio di programmazione, ma apprende anche alcune strutture logiche come cicli, definizioni di funzioni e schemi di dichiarazione, mostrando come sia possibile acquisire una comprensione strutturale attraverso i dati sequenziali.

8.4 Searching for Interpretable Cells

È possibile visualizzare ciò che una rete apprende analizzando i valori del vettore nascosto. Ogni neurone del vettore può infatti specializzarsi nel rilevare una certa caratteristica. Dopo il passaggio attraverso la funzione lineare e l'attivazione, ogni componente del vettore assume valori compresi tra -1 e 1. Questi valori possono essere rappresentati con una codifica cromatica, in modo da evidenziare se un neurone sta attivandosi in risposta a una determinata struttura.

Alcuni neuroni possono rispondere a pattern specifici come la profondità dei blocchi di codice (es. indentazione), la presenza di commenti o virgolette, oppure il fatto che ci si trovi all'interno di una struttura condizionale (if). Queste attivazioni visivamente interpretabili permettono di indagare se la rete ha imparato rappresentazioni interne coerenti con strutture logiche del dato.

8.5 Image Captioning

Concetti simili possono essere applicati al dominio visivo. Utilizzando una CNN, è possibile estrarre un vettore di caratteristiche da un'immagine (ad esempio tramite un modello preaddestrato su ImageNet), che viene poi fornito a una RNN per generare una sequenza di parole che descriva il contenuto dell'immagine.

La CNN si occupa di codificare aspetti come forma, oggetti e texture, mentre la RNN riceve questo embedding visivo come contesto iniziale e produce la didascalia parola per parola. Il processo è analogo alla generazione di testo: si parte con un token di inizio sequenza e si procede passo dopo passo, aggiornando lo stato nascosto e campionando la parola successiva.

L'architettura finale può includere componenti che combinano il contesto visivo con lo stato nascosto, migliorando la coerenza e la pertinenza della descrizione generata rispetto all'immagine di partenza.

8.6 Vanilla RNN Gradient Flow

Il flusso del gradiente nelle RNN standard è soggetto a problemi noti come l'esplosione o la scomparsa del gradiente. Questo avviene perché la derivata della funzione di attivazione (come $\tanh$) viene moltiplicata ripetutamente per sé stessa lungo la catena temporale.

Quando il valore singolare massimo dei pesi è maggiore di 1, si può avere un'esplosione del gradiente, che rende instabile l'addestramento. Questo problema viene affrontato attraverso tecniche come il **gradient clipping**, che consiste nel ridurre la norma del gradiente se supera una certa soglia. Al contrario, se il valore singolare massimo è inferiore a 1, il gradiente tende a svanire, rendendo difficile l'apprendimento delle dipendenze a lungo termine. In tal caso, conviene adottare architetture alternative come le LSTM.

8.7 Long Short-Term Memory (LSTM)

Le LSTM risolvono il problema della memoria a lungo termine grazie a una struttura interna composta da più "gate" che controllano il flusso delle informazioni: il **forget gate** decide cosa cancellare, l'**input gate** cosa scrivere, il **gate gate** (o candidate gate) quanto scrivere, e l'**output gate** cosa restituire all'esterno. Questi meccanismi regolano dinamicamente il contenuto della cella di memoria.

Il flusso del gradiente all'interno di una LSTM è preservato più a lungo perché il backpropagation attraverso la cella c_t comporta solo moltiplicazioni elemento per elemento, senza passaggi attraverso matrici di pesi, rendendo la retropropagazione più stabile, in maniera analoga a quanto avviene nelle ResNet.

8.8 Single-layer e Multi-layer RNNs

Una RNN può essere composta da un singolo strato oppure da più strati. Nelle **single-layer RNN**, lo stato nascosto viene aggiornato in ogni istante temporale e passato direttamente all'output. Nelle **multi-layer RNN**, l'output di uno strato serve da input per lo strato successivo, aumentando la capacità di modellare strutture complesse.

La formulazione si adatta anche alle LSTM, dove la profondità delle celle viene organizzata a livelli successivi, ognuno con le proprie porte e stati interni.

8.9 Altre Varianti di RNN

Oltre alle LSTM, esistono architetture alternative come le **GRU** (Gated Recurrent Unit), che unificano alcune delle porte delle LSTM in una struttura più compatta ma efficace. Le GRU

usano due gate: uno di reset e uno di aggiornamento, che combinano le funzioni delle porte di input e forget.

Numerose ricerche esplorano anche varianti architetturali empiriche per ottimizzare la rappresentazione delle sequenze, ad esempio nella traduzione automatica statistica. Alcune architetture sono progettate per facilitare l'apprendimento di frasi o strutture ripetitive, migliorando l'efficienza dell'addestramento.

8.10 Conclusioni

Le RNN offrono una grande flessibilità progettuale, ma le versioni standard (vanilla RNN) non sono adatte a compiti complessi. Le architetture con interazioni additive, come LSTM e GRU, sono più robuste e facilitano l'apprendimento grazie a una migliore gestione del gradiente. Esistono tecniche per limitare i problemi di esplosione e scomparsa del gradiente, ma la ricerca continua su architetture più semplici e performanti, sia dal punto di vista teorico che empirico.

9 Object Detection e Multi-Task Learning

Nel campo della visione artificiale, la rilevazione di oggetti (object detection) si distingue da compiti come la classificazione o la segmentazione semantica per il fatto che non solo identifica **cosa** è presente in un'immagine, ma anche **dove** si trova. Il modello prende in input un'immagine RGB e restituisce un insieme di oggetti rilevati, ognuno descritto da:

- una **categoria** (scelta da un insieme fisso e noto di classi),
- un **bounding box**, ovvero quattro valori numerici che definiscono la posizione (x, y) e le dimensioni (larghezza e altezza) del riquadro.

Le principali difficoltà di questo compito includono:

- La presenza di un **numero variabile di oggetti** in ciascuna immagine, che implica la necessità di produrre un output flessibile.
- L'output combinato di **tipi diversi di informazione**, ovvero sia etichette di classe che coordinate spaziali.
- La dimensione delle immagini: se la classificazione lavora tipicamente su input 224x224, per la detection si richiede maggiore risoluzione (spesso 800x600 o più).

9.1 Multi-Task Learning (MTL)

Nel contesto di sistemi complessi, è frequente la presenza di **più task supervisionati** da affrontare in parallelo. L'obiettivo del **Multi-Task Learning** è quello di risolverli contemporaneamente sfruttando una rappresentazione condivisa. Questo permette un trasferimento di conoscenza tra i task che può migliorare la generalizzazione.

Formalmente, si considera un insieme di T compiti supervisionati, ciascuno con i propri dati di addestramento D_t e un vettore di parametri w_t . La funzione di perdita complessiva da minimizzare è:

$$\mathcal{L}_{\text{MTL}} = \sum_{t=1}^T \lambda_t \mathcal{L}_t(D_t, w_t) + \alpha \|W\|_F^2$$

dove:

- W è la matrice i cui vettori colonna sono i parametri w_t di ciascun task,

- $\|W\|_F^2$ è la **norma di Frobenius**, definita come $\|W\|_F^2 = \sum_{t=1}^T \|w_t\|_2^2$, che equivale a una regolarizzazione ℓ_2 su ciascun vettore di parametri,
- α è un iperparametro che controlla l'intensità della regolarizzazione.

L'ottimizzazione può anche essere formulata utilizzando la **trace norm** $|W|_*$. Minimizzarla ha l'effetto di incoraggiare una **soluzione a bassa complessità (low-rank)**, cioè una rappresentazione condivisa tra i task.

Questo approccio è noto come **Multi-Task Feature Learning (MTFL)**, e può essere esteso anche a classificatori non lineari mediante il mapping delle feature in uno spazio kernel $\phi(x)$.

Il vantaggio è duplice: da una parte si ottiene una **regolarizzazione efficace** che migliora la generalizzazione, dall'altra si promuove un **trasferimento bidirezionale** tra i task, evitando situazioni in cui solo uno dei compiti beneficia degli altri.

Infine, se non vi è correlazione tra i compiti, può essere più utile trattarli separatamente.

9.2 Structured Output Learning

Questo approccio considera dati strutturati, cioè insiemi di elementi che non sono indipendenti, ma presentano relazioni reciproche. Alcuni esempi includono documenti ipertestuali, immagini con annotazioni strutturate o strutture chimiche.

Structured Data: dati in cui l'informazione è contenuta sia nei singoli elementi, sia nelle relazioni tra di essi.

L'obiettivo del structured output learning è prevedere risultati che rispettino tali relazioni, anziché trattare ogni output come indipendente dagli altri.

9.3 Transfer Learning

Il Transfer Learning si basa sull'idea di riutilizzare la conoscenza appresa in un dominio (ad esempio da **ImageNet**) per risolvere un compito diverso. L'intuizione si può rendere con la metafora del film *Karate Kid* ("Wax on, wax off"), dove un'abilità appresa in un contesto viene sfruttata in un altro, all'apparenza non correlato.

Principio chiave: la conoscenza non è confinata al dominio in cui è stata appresa, ma può essere trasferita e riutilizzata in altri problemi.

9.3.1 Structured Output: localizzazione come regressione

Per rilevare un singolo oggetto, si può considerare la localizzazione come un **problema di regressione**: si calcola un vettore che contiene le coordinate del bounding box (x', y', w', h') e lo si confronta con il box corretto mediante **L2 loss**. La classificazione, invece, viene trattata con una **softmax loss**. La perdita totale è una **multitask loss**: una somma pesata tra le due.

9.4 Multiple object detection

Nel caso di immagini con oggetti multipli, il numero di output necessari varia da immagine a immagine, rendendo il problema più complesso. Serve quindi un approccio flessibile, capace di produrre un numero arbitrario di bounding box.

9.4.1 Sliding Window

Un approccio iniziale all'object detection è quello dello **sliding window**: si applica una CNN a molte sotto-regioni dell'immagine (crops), trattando ciascuna come una possibile finestra contenente un oggetto. La CNN classifica ciascuna finestra come oggetto o sfondo.

Questo metodo, però, richiede valutare un numero elevatissimo di regioni: ad esempio, su un'immagine 800x600 si possono generare decine di milioni di possibili box.

9.4.2 R-CNN e Region Proposal

Per rendere il processo più efficiente, l'approccio **R-CNN** propone di:

1. Identificare **regioni dell'immagine** che hanno maggiore probabilità di contenere oggetti (region proposals), ad esempio tramite **Selective Search**.
2. Ridimensionare ciascuna regione a 224×224 e passarla attraverso una CNN per classificazione e regressione del bounding box.
3. Scegliere un **sottoinsieme** delle regioni proposte usando punteggi di confidenza (es. soglia per categoria o top-K).
4. Confrontare i risultati con i *ground truth*.

9.5 Intersection over Union (IoU)

Per valutare quanto un box predetto sia accurato rispetto a uno reale, si utilizza la metrica **Intersection over Union (IoU)**, anche nota come indice di Jaccard:

Valori tipici:

- $\text{IoU} > 0.5 \rightarrow$ "decente"
- $\text{IoU} > 0.7 \rightarrow$ "piuttosto buono"
- $\text{IoU} > 0.9 \rightarrow$ "quasi perfetto"

Questo criterio è fondamentale per valutare il successo dei metodi di detection rispetto alle ground truth.

9.6 Non-Maximum Suppression (NMS)

Spesso, quando il modello individua un oggetto (es. una persona), non restituisce una sola box, ma più box sovrapposte, tutte vicine e tutte con punteggi alti, ma leggermente diversi.

Questo succede perché la rete scorre su regioni sovrapposte e fa più predizioni molto simili. Nel risultato finale non si vogliono 5 box per la stessa persona, ma solo una.

In presenza di molte predizioni sovrapposte, è necessario un processo di **soppressione dei duplicati**. L'algoritmo di Non-Max Suppression (NMS) seleziona iterativamente la box con il punteggio più alto ed elimina tutte le box con IoU superiore a una soglia (es. 0.7).

Passaggi dell'algoritmo:

1. Seleziona la box con il punteggio più alto
2. Elimina le box che hanno IoU elevato con essa
3. Ripeti finché restano box

Tuttavia, questo metodo può eliminare anche predizioni corrette nel caso di oggetti molto ravvicinati.

9.7 Valutazione dei Rilevatori: Mean Average Precision (mAP)

Per valutare le prestazioni di un object detector, si utilizza la metrica **mean Average Precision (mAP)**. I passaggi sono:

1. Eseguire il detector su tutte le immagini (con NMS)
2. Per ogni categoria:
 - Ordinare le predizioni per confidenza (dal più alto al più basso)

- Per ciascuna predizione:
 - Se ha $\text{IoU} > 0.5$ con un ground truth, è positiva (e si elimina quel GT)
 - Altrimenti è negativa
 - Si costruisce la **Precision-Recall curve**, e si calcola l'**AP** = area sotto la curva
3. Calcolare la media degli AP per ottenere il **mAP**:
 - es. Car = 0.65, Cat = 0.80, Dog = 0.86 $\rightarrow$ mAP = 0.77
 4. Per il benchmark **COCO**, si calcola mAP per varie soglie di IoU (da 0.5 a 0.95 a intervalli di 0.05), e si fa la media:
 - es. mAP@0.5 = 0.77, mAP@0.55 = 0.71, ..., mAP@0.95 = 0.20 $\rightarrow$ COCO mAP = 0.4

Questa metrica è oggi lo standard principale per confrontare diversi algoritmi di object detection.

9.8 Fast R-CNN

Il problema principale di R-CNN è la **lentezza**: bisogna eseguire migliaia di forward pass per ogni immagine. **Fast R-CNN** risolve questo eseguendo la CNN una sola volta sull'intera immagine.

Si utilizza una rete backbone (AlexNet, VGG, ResNet...) per estrarre una mappa di feature dall'intera immagine. Da questa mappa si **croppano** le regioni di interesse (RoI), che vengono poi elaborate da una rete leggera (fully connected) per classificazione e regressione.

Esempi

- Con **AlexNet**, i primi 5 strati convoluzionali sono usati per la backbone, i restanti FC per classificazione per regione.
- Con **ResNet**, si usa solo l'ultimo blocco come rete per-region, il resto è parte della backbone.

9.9 Cropping Features: RoI Pool vs RoI Align

- **RoI Pooling**: proietta le regioni proposte sulla mappa di feature e le suddivide in sotto-regioni (es. 2x2). Per ogni sotto-regione si applica un max-pooling. Questo introduce errori di **allineamento** (snapping a griglia).
- **RoI Align**: elimina lo snapping. Divide la regione in sotto-regioni equidistanti, anche se non allineate alla griglia. Le feature sono campionate con **interpolazione bilineare** e combinate linearmente. Questo metodo è **differenziabile** e più preciso per localizzazione.

La RoI Align ha migliorato significativamente la precisione nei modelli successivi come Mask R-CNN.

9.10 Faster R-CNN: Regional Proposal Network (RPN)

Il principale collo di bottiglia di Fast R-CNN è rappresentato dal metodo di **Selective Search** utilizzato per generare le regioni proposte (RoIs), operazione eseguita sulla CPU e molto lenta.

Faster R-CNN migliora ulteriormente l'efficienza introducendo una **Regional Proposal Network (RPN)**: un modulo neurale che impara a generare le proposte direttamente dalle feature estratte dalla CNN backbone.

Il processo avviene in due fasi:

1. Stage 1: per immagine

Si esegue la CNN backbone su tutta l'immagine.

La RPN produce le regioni proposte a partire dalle feature estratte.

2. Stage 2: per regione proposta

Si croppano le feature relative a ogni regione con RoI Align.

Si classificano le proposte e si predicono i bounding box refined.

La RPN esamina ogni punto nella mappa delle feature e considera **K anchor box** di diverse forme e dimensioni, predicendo:

- se ogni anchor contiene un oggetto (classificazione binaria);
- come trasformare l'anchor per approssimare meglio il box vero (regressione).

9.11 Perdita (Loss) di Faster R-CNN

Faster R-CNN viene addestrato con la combinazione di 4 funzioni obiettivo:

1. **RPN classification**: classificazione binaria degli anchor (oggetto/sfondo);
2. **RPN regression**: regressione per adattare l'anchor al box oggetto;
3. **Object classification**: classificazione tra le varie categorie di oggetto;
4. **Object regression**: ulteriore regressione del box proposta → box finale.

9.12 Caratteristiche chiave

- È un **rilevatore a due stadi** (two-stage detector);
- Permette una maggiore accuratezza rispetto ai metodi one-stage;
- Ha costi computazionali comunque inferiori rispetto a R-CNN.

9.13 Riepilogo delle differenze

- **R-CNN**: esegue la CNN su ogni regione → molto lento.
- **Fast R-CNN**: esegue la CNN una volta sola → cropping differenziabile.
- **Faster R-CNN**: elimina anche Selective Search, rendendo il processo end-to-end con RPN.
- **Single-stage detector**: come SSD, YOLO → tutto avviene in un unico passaggio convoluzionale.

9.14 Considerazioni finali

- I metodi a due stadi (Faster R-CNN) raggiungono la miglior accuratezza, ma sono più lenti.
- I metodi one-stage (come SSD) sono più veloci ma meno accurati.
- L'uso di backbone più potenti (es. ResNeXt, Feature Pyramid Networks) migliora le performance.
- Si ottengono ulteriori miglioramenti con:
 - **Training prolungato**
 - **Augmentation** nel test-time
 - **Ensembling** e utilizzo di molti dati

In sintesi, la rilevazione di oggetti ha evoluto la sua pipeline da metodi lenti e rigidi a sistemi completamente end-to-end, bilanciando precisione ed efficienza attraverso l'integrazione tra reti convoluzionali e strategie di proposta apprese.

9.15 Semantic Segmentation

La **segmentazione semantica** ha come obiettivo quello di assegnare una classe a **ogni pixel** di un'immagine. A differenza della detection, qui non si individuano oggetti con bounding box, ma si classificano direttamente le aree visive — ad esempio: cielo, erba, gatto.

La segmentazione semantica non distingue tra istanze della stessa classe (es. due gatti vengono considerati entrambi come "gatto").

9.15.1 Sliding Window

Un approccio iniziale alla segmentazione semantica consiste nell'applicare una CNN a piccole porzioni dell'immagine (patch). Si:

1. **Estrae una patch** intorno a un pixel centrale;
2. **Classifica il pixel centrale** passando la patch attraverso una CNN, che restituisce una classe (es. mucca, erba).

Questo metodo funziona, ma presenta gravi **limiti di efficienza**: molte patch si sovrappongono, e le feature condivise vengono ricalcolate più volte. Questo porta a ridondanze e a un uso inefficiente delle risorse computazionali.

9.15.2 Fully Convolutional Networks (FCN)

Una soluzione più elegante e moderna è progettare una rete **fully convolutional**, ovvero composta solo da layer convoluzionali, senza dense layer finali.

L'idea è:

- Prendere l'immagine intera come input;
- Farla passare attraverso più strati convoluzionali con stride adeguati;
- Ottenere una mappa (tensor) in output con **le stesse dimensioni spaziali dell'input**;
- Per ogni posizione (i, j) della mappa, applicare una **argmax** sul vettore delle classi predette, ottenendo la classe assegnata a quel pixel.

9.15.3 Formulazione

- L'output della rete ha forma $C \times H \times W$, dove C è il numero di classi;
- Applicando argmax lungo C otteniamo una mappa $H \times W$ di etichette di classe.

9.15.4 Training della rete

La rete viene addestrata usando una **loss per pixel**, di solito la **cross-entropy**, confrontando l'output della rete con la ground truth pixel-wise.

In pratica, è come se ogni pixel fosse un esempio di classificazione supervisionata.

9.15.5 Vantaggi e sfide

- Le reti fully convolutional sono molto più efficienti rispetto allo sliding window;
- Tuttavia, sorgono **problemi dovuti alla perdita di risoluzione** causata da strati con stride o pooling;
- La struttura finale deve garantire la **ricostruzione precisa della segmentazione**, mantenendo la corrispondenza tra input e output.

La segmentazione semantica è dunque un compito fondamentale nella visione artificiale, soprattutto per scenari dove conta **la comprensione del contesto spaziale** (es. guida autonoma, robotica, diagnostica medica).

9.16 Problemi nella segmentazione semantica

Un **primo** problema nella segmentazione fully convolutional riguarda il **campo ricettivo** (receptive field). Il campo ricettivo effettivo di una rete cresce **linearmente con la profondità** (numero di layer convoluzionali):

- 2 layer $3 \times 3 \rightarrow$ receptive field 5×5
- 3 layer $3 \times 3 \rightarrow$ receptive field 7×7

Per assegnare un'etichetta a un singolo pixel **considerando l'intero oggetto**, la rete dovrebbe avere un campo ricettivo molto grande. Questo richiederebbe molti layer, rendendo il modello **computazionalmente costoso**.

Il valore predetto per un pixel dipende da una porzione dell'immagine di input sempre più ampia, man mano che si procede negli strati della rete.

A livello pratico, la label di un pixel **dipende dal contesto**, cioè da tutti i pixel circostanti. Per ottenere un campo ricettivo che copra tutta l'immagine bisognerebbe quindi aumentare molto la profondità della rete — ma questo non è efficiente.

Il **secondo** problema è che l'applicazione della convoluzione a immagini ad alta risoluzione è molto costosa. Ad esempio, il backbone di ResNet inizia con uno **stem** che downscala aggressivamente l'immagine per ridurre il carico computazionale.

9.17 Strategia Fully Convolutional: Downsampling e Upsampling

Per affrontare questi problemi si utilizza una **struttura encoder-decoder**:

- **Fase di compressione (downsampling)**: si riducono progressivamente le dimensioni spaziali, aumentando la profondità dei canali. Si ottiene una rappresentazione compatta e ricca di contenuto semantico.
- **Fase di espansione (upsampling)**: si riportano le dimensioni spaziali originali, costruendo una mappa predittiva pixel-wise.

Questo approccio permette di:

1. Catturare il **contesto globale** dell'immagine;
2. Evitare di mantenere feature map giganti in tutte le fasi, risparmiando memoria e calcolo.

9.17.1 Downsampling

È ben noto come eseguire il downsampling: pooling, convoluzioni con stride > 1 , ecc. È stato applicato in modelli come AlexNet, VGG, ecc.

9.17.2 Upsampling

È più sfidante, perché le architetture classiche non includevano meccanismi espliciti per risalire di risoluzione. Alcune tecniche sono:

1. **Unpooling**
Variante del max pooling che memorizza la posizione del valore massimo e lo ripristina nella stessa posizione durante l'upsampling.
2. **Interpolazioni**
 - **Nearest neighbor**: duplica i valori in posizioni vicine.
 - **Bilinear**: interpola linearmente tra i pixel adiacenti (2D).

- **Bicubic**: utilizza 3 pixel vicini in x e y per interpolazione cubica (tipico resize di immagini).

3. Transposed Convolution (learnable upsampling)

Nota anche come:

- deconvolution (termine improprio),
- upconvolution,
- fractionally strided convolution,
- backward convolution.

Funziona come una **convoluzione inversa**: anziché prendere un filtro e farlo scorrere sull'input per produrre un output più piccolo, qui si genera un output più grande.

Esempio:

- **Conv 3×3 , stride 2, pad 1**: da $4 \times 4 \rightarrow 2 \times 2$
- **Transpose Conv 3×3 , stride 2, pad 1**: da $2 \times 2 \rightarrow 4 \times 4$ (con overlapping)

In 1D:

Ogni valore viene moltiplicato per un filtro e distribuito nello spazio di output, le sovrapposizioni vengono sommate.

Spesso serve tagliare un pixel ai bordi per riportare la dimensione desiderata.

Questo metodo è **apprendibile** e più flessibile rispetto all'interpolazione, ma può introdurre artefatti (checkerboard effect) se non correttamente implementato.

In sintesi, il passaggio da sliding window a modelli fully convolutional encoder-decoder ha migliorato enormemente l'efficienza e la qualità della segmentazione semantica, bilanciando precisione, contesto e costi computazionali.

9.18 Computer Vision Tasks: Detection, Segmentation e Istanza

Nel campo della computer vision esistono diversi task fondamentali per comprendere e interpretare le immagini. I più rilevanti sono:

- **Object Detection**: identifica e localizza gli oggetti tramite bounding box, assegnando ad ogni riquadro una categoria.
- **Semantic Segmentation**: assegna un'etichetta di classe a ciascun pixel, ma **non distingue tra diverse istanze** della stessa classe.

Per chiarire meglio questa distinzione, si introduce il concetto tra:

- **Things**: oggetti distinti (es. gatti, automobili, persone).
- **Stuff**: superfici o materiali continui (es. cielo, acqua, erba) che non si segmentano in istanze.

9.18.1 Instance Segmentation

La **instance segmentation** combina i vantaggi della detection e della segmentazione semantica. Mira a:

- rilevare tutti gli oggetti presenti,
- assegnare un **maschera pixel-wise** a ogni **istanza distinta**.

In altre parole, non solo dice “questi pixel sono un cane”, ma anche “questi pixel sono il primo cane, e questi altri sono il secondo”.

9.18.2 Approccio classico: Mask R-CNN

Il metodo più noto è **Mask R-CNN**, che estende Faster R-CNN con una testa aggiuntiva per la segmentazione:

1. **Object detection** (tramite Faster R-CNN):

Estrazione di region proposals (RPN);
Classificazione e regressione dei bounding box;

2. **Segmentazione per istanza**:

Per ogni proposta, si predice una maschera di segmentazione dettagliata.

La testa per la segmentazione opera su feature allineate (con **RoI Align**) e predice una maschera binaria 28×28 per ogni classe, con convoluzioni dedicate.

9.18.3 Architettura

- Per ogni regione proposta, si calcola:
 - la classe dell'oggetto;
 - la posizione precisa del box;
 - e la maschera che ne segmenta l'area interna.

9.18.4 Output

Il risultato è una mappa che mostra:

- bounding box e categorie come nella detection;
- segmentazione pixel-wise **per ciascuna istanza** (colori diversi per oggetti diversi della stessa classe).

Questa architettura consente una comprensione ricca e dettagliata della scena, utile per applicazioni come guida autonoma, robotica, annotazione avanzata o realtà aumentata.

9.19 Panoptic Segmentation

Abbiamo visto che:

- la **semantic segmentation** assegna etichette a tutti i pixel, distinguendo tra “things” e “stuff”, ma **non separa le istanze**;
- la **instance segmentation** identifica solo i “things” e **li distingue tra loro**.

La **panoptic segmentation** unisce entrambi gli approcci:

- assegna **un'etichetta di classe a ogni pixel**;
- separa anche **le istanze per le categorie “thing”**.

In questo modo, si ottiene una mappa completa e coerente dell'immagine, segmentando sia oggetti distinti sia superfici continue.

9.20 Keypoints Estimation

Un altro task interessante è la **stima dei keypoint umani**, ovvero la localizzazione dei punti chiave del corpo umano (es. naso, occhi, gomiti, spalle...). Un tipico modello prevede 17 key-points:

- naso, occhi, orecchie, spalle, gomiti, polsi, anche, ginocchia, caviglie (sinistra e destra).

Questo approccio viene esteso anche ad altri compiti, come la generazione di **caption per bounding box**.

9.20.1 Captioning e Visual Genome

Si può aggiungere una **testa ricorrente (RNN)** a modelli come Faster R-CNN per generare **una descrizione testuale** per ogni bounding box. Questo porta a modelli che non solo rilevano oggetti, ma li **descrivono con frasi naturali**.

Un dataset chiave per questo è **Visual Genome**, sviluppato da Fei-Fei Li, che include:

- bounding boxes;
- caption per ogni box;
- **relazioni semantiche** tra gli oggetti (es. “la persona è sopra l’elefante”).

Questo arricchisce enormemente il livello di comprensione semantica di un’immagine.

9.21 3D Shape Prediction

Il modello può essere ulteriormente esteso con una testa che predice **mesh 3D**:

- una mesh è una rappresentazione 3D di un oggetto tramite triangoli nello spazio.
- serve un dataset che includa immagini e forme 3D corrispondenti (ground truth 3D).

Il risultato è un sistema in grado di:

- localizzare oggetti (detection);
- segmentarli (mask);
- descriverli (caption);
- stimarne la **forma tridimensionale**.

Questo approccio modulare con teste multiple rende il sistema estremamente versatile e permette una **comprensione profonda dell’immagine da più punti di vista**, sia 2D sia 3D.

—

10 Unsupervised e Self-Supervised Learning

10.1 Supervised Learning

Nel contesto del supervised learning, i dati disponibili per l’addestramento sono costituiti da coppie (x, y) , dove x rappresenta un input (ad esempio un’immagine) e y è l’etichetta associata (come una categoria o un valore numerico). L’obiettivo principale in questo paradigma di apprendimento è quello di apprendere una funzione che riesca a mappare l’input x all’output desiderato y .

Questa funzione appresa può poi essere utilizzata per effettuare previsioni su nuovi dati non visti. Alcuni esempi comuni di problemi trattati con supervised learning sono:

- Classificazione (es. riconoscere se un’immagine contiene un gatto o un cane)
- Regressione (es. prevedere il prezzo di una casa dato un insieme di caratteristiche)
- Object detection, semantic segmentation, image captioning

Tuttavia, l’approccio supervisionato richiede dataset etichettati, il che comporta un costo significativo soprattutto quando le etichette devono essere assegnate da esperti (come nel settore medico).

10.2 Unsupervised Learning

Nell’unsupervised learning, si dispone unicamente dei dati x , privi di etichette. Questo rende l’acquisizione del dataset molto più economica, ma pone nuove sfide: l’obiettivo ora è individuare

delle strutture nascoste all'interno dei dati stessi, senza alcuna guida esterna.

L'idea è che i modelli possano apprendere caratteristiche utili osservando solo i dati grezzi. In questo contesto, alcune tecniche comuni includono:

- Clustering
- Riduzione di dimensionalità
- Feature learning
- Density estimation

Queste tecniche sono fondamentali per comprendere la struttura del mondo visivo e per pre-processare i dati in contesti più complessi.

10.3 Self-Supervised Learning

Il self-supervised learning rappresenta un paradigma emergente nel campo del machine learning. Nasce con l'intento di superare alcune delle limitazioni tipiche del supervised learning, in particolare la necessità di avere un dataset etichettato per ogni compito specifico. Etichettare i dati è un processo costoso e, in molti contesti (ad esempio quello medico), anche poco praticabile a causa della necessità di annotazioni da parte di esperti.

Contemporaneamente, esiste una grande abbondanza di dati non etichettati. Piattaforme come Facebook o YouTube generano miliardi di immagini o ore di contenuti video ogni giorno. Questo rappresenta un'opportunità enorme per lo sviluppo di nuove metodologie di apprendimento automatico.

Il self-supervised learning permette ai modelli di apprendere rappresentazioni significative a partire da dati non etichettati. Sebbene tecnicamente sia una forma di apprendimento non supervisionato, in questo caso è il dato stesso a fornire la supervisione.

In pratica, si nasconde una parte del dato e si progetta un compito (proxy task) che consiste nel prevedere l'informazione mancante. Questo induce una funzione di perdita che guida il modello a imparare caratteristiche semantiche utili, riutilizzabili anche per altri compiti.

Un tipico framework di self-supervised learning è costituito da due fasi:

1. Pretraining: il modello viene addestrato su un compito ausiliario (pretext task) per imparare rappresentazioni generiche.
2. Fine-tuning: le rappresentazioni apprese vengono poi riutilizzate per risolvere un compito reale (downstream task) in cui i dati etichettati sono pochi.

Questo approccio consente di sfruttare in modo efficiente grandi quantità di dati non etichettati, riducendo la dipendenza da annotazioni costose e migliorando la performance su task specifici.

I principi alla base del self-supervised learning ricordano il modo in cui imparano gli esseri umani, in particolare durante l'infanzia: attraverso l'osservazione e l'inferenza di relazioni implicite, senza una guida esplicita.

Un esempio classico di proxy task è la relative positioning, in cui il modello riceve due porzioni della stessa immagine e deve predire la loro posizione relativa. Questo induce il modello a catturare la struttura spaziale e semantica dell'immagine, senza bisogno di etichette.

Il self-supervised learning apre la strada a modelli più adattabili, scalabili e meno dipendenti da processi di annotazione manuale.

10.4 Generative Models

I modelli generativi sono modelli di apprendimento automatico che, dati degli esempi di addestramento, sono in grado di generare nuovi dati che provengano dalla stessa distribuzione dei dati osservati.

In termini probabilistici, se i dati osservati provengono da una distribuzione $p_{\text{model}}(x)$, l'obiettivo di un modello generativo è imparare una distribuzione $p_{\text{model}}(x)$ che sia il più simile possibile a $p_{\text{data}}(x)$. Una volta addestrato, il modello è in grado di generare nuovi campioni (es. immagini, suoni, testi) che appaiono realistici e coerenti con i dati reali.

10.5 Density Estimation nei Modelli Generativi

Un problema fondamentale nell'apprendimento non supervisionato è la stima della densità dei dati, ovvero la modellazione della probabilità $p(x)$. Questo compito può essere affrontato in due modi principali:

- **Explicit Density Estimation:** si definisce esplicitamente la forma della distribuzione $p_{\text{model}}(x)$ e si risolve per essa. In questo approccio il modello è in grado di calcolare la probabilità esatta di un dato.
- **Implicit Density Estimation:** si costruisce un modello in grado di generare campioni da $p_{\text{model}}(x)$, senza però fornire esplicitamente una formula o un valore di densità.

10.5.1 Tipologie di Modelli Generativi

1. Explicit Density Models

- **Tractable Density:** modelli che possono calcolare esattamente $p(x)$. Esempi:
 - Autoregressive models
 - NADE / MADE
 - NICE / RealNVP
 - Glow
 - FFJORD
- **Approximate Density:** modelli che possono approssimare $p(x)$. Esempi:
 - Variational Autoencoders (VAE)
 - Boltzmann Machines (con catene di Markov)

2. Implicit Density Models

- **Markov Chain-based:**
Generative Stochastic Networks (GSN)
- **Direct:**
Generative Adversarial Networks (GANs), che imparano a generare dati realistici tramite un meccanismo di competizione tra due reti (generatore e discriminatore)

I modelli impliciti non calcolano la probabilità $p(x)$ ma possono generare nuovi dati che risultano coerenti con quelli reali.

Questo schema permette di classificare i modelli generativi a seconda del modo in cui trattano la densità e del livello di accesso alla funzione $p(x)$.

10.6 Explicit Density Estimation

L'obiettivo dell'explicit density estimation è scrivere una funzione esplicita per $p(x)=f(x,W)$, dove W rappresenta i parametri del modello. Dato un dataset $x(1),x(2),\dots,x(N)$, il modello viene addestrato massimizzando la probabilità dei dati di training:

$$W^* = \arg \max_W \prod_i p(x^{(i)}) = \arg \max_W \sum_i \log p(x^{(i)}) = \arg \max_W \sum_i \log f(x^{(i)}, W)$$

Supponendo che x sia costituito da più sottoparti $x=(x_1, x_2, \dots, x_T)$, è possibile applicare la regola della catena per decomporre la probabilità: $p(x) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1})$

Questa formulazione viene applicata anche nel caso delle immagini, dove ogni pixel può essere considerato come una componente della variabile x . Si ottiene: $p(x) = \prod_{i=1}^{n^2} p(x_i | x_1, \dots, x_{i-1})$

10.7 Pixel-RNN

Il modello **PixelRNN** genera immagini pixel per pixel partendo da un angolo. Utilizza una rete ricorrente (RNN) per modellare le dipendenze tra i pixel precedenti. Per ogni pixel (x, y) , si calcola uno stato nascosto che dipende dai valori RGB e dagli stati nascosti dei pixel sopra e a sinistra: $h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$

A ogni pixel si predice prima il canale rosso, poi il blu, poi il verde. Si utilizza una softmax su 256 valori (da 0 a 255). Questo approccio è efficace ma ha lo svantaggio di essere lento, poiché la generazione è sequenziale.

10.8 Pixel-CNN e Gated PixelCNN

Le immagini possiedono una struttura bidimensionale (2D) intrinseca. Appiattirle in vettori unidimensionali (1D) per il processamento, come fanno i modelli autoregressivi classici, può compromettere la cattura delle relazioni spaziali. Il **PixelCNN** sfrutta le convoluzioni per rispettare questa struttura, introducendo **convoluzioni mascherate** per mantenere l'autoregressività.

La mascheratura impedisce che l'informazione fluisca dai pixel futuri (non ancora generati) verso quelli correnti, rispettando così l'ordine imposto. Questo garantisce che la previsione di ogni pixel dipenda solo dai pixel precedenti.

Tuttavia, la prima versione del PixelCNN presentava limitazioni: lentezza nell'addestramento e difficoltà nel catturare efficacemente le dipendenze spaziali.

10.8.1 Gated PixelCNN

Il **Gated PixelCNN** migliora questo approccio introducendo due flussi convoluzionali (verticale e orizzontale), per una modellazione più efficace delle dipendenze tra pixel.

Vantaggi:

1. **Modellazione delle dipendenze attraverso regioni di contesto:**
ogni pixel viene previsto considerando un contesto spaziale più ampio, migliorando la capacità del modello di catturare relazioni complesse.
2. **Addestramento parallelo:**
sebbene la generazione rimanga sequenziale, durante l'addestramento è possibile parallelizzare perché i pixel del contesto sono già noti. Questo riduce significativamente i tempi rispetto a modelli come PixelRNN.
3. **Generazione sequenziale:**
nonostante l'efficienza nell'addestramento, la generazione resta sequenziale, il che rappresenta un collo di bottleneck per immagini ad alta risoluzione.

10.8.2 Riflessioni su PixelCNN

Le convoluzioni mascherate e architetture come Gated PixelCNN pongono in evidenza un compromesso: ottima qualità di generazione a scapito della velocità. Sono modelli efficaci per catturare dipendenze dettagliate, ma la generazione sequenziale limita l'applicabilità in contesti real-time o di grande scala.

Vantaggi di PixelCNN:

1. **Calcolo esplicito della likelihood:** consente di valutare matematicamente la qualità del modello.
2. **Valutazione tramite likelihood:** permette confronti diretti tra modelli.
3. **Campioni di alta qualità:** genera immagini consistenti con la distribuzione appresa.

Limiti:

- **Generazione sequenziale:** ogni pixel è generato uno alla volta, rallentando il processo.

Per superare questi limiti, sono stati introdotti miglioramenti come:

- Gated convolutions
- Short-cut connections
- Discretized logistic loss
- Architetture multi-scale
- Tecniche di training avanzate

Infine, PixelCNN definisce una funzione di densità trattabile e ottimizza direttamente la likelihood dei dati di training:

$$p_{\theta}(x) = \prod_{i=1}^{n^2} p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

Al contrario, i **Variational Autoencoders (VAE)** definiscono una densità intrattabile tramite una variabile latente z :

$$p(x) = \int p(z) p(x | z) dz$$

Non è possibile ottimizzare direttamente questa espressione, per cui si ottimizza un bound inferiore della likelihood (Evidence Lower Bound, ELBO).

10.9 Autoencoder

Gli autoencoder sono modelli non supervisionati che imparano a comprimere e ricostruire i dati. Sono composti da:

- **Encoder:** comprime i dati in uno spazio latente
- **Decoder:** ricostruisce i dati a partire dallo spazio latente

Non richiedono etichette. L'obiettivo è che lo spazio latente contenga informazioni rilevanti e compatte.

Possono essere usati per:

- Ridurre la dimensionalità
- Pre-addestrare reti neurali su dati non etichettati

10.10 Variational Autoencoders (VAE)

I VAE introducono una prospettiva probabilistica agli autoencoder. Invece di mappare ogni input a un singolo punto nello spazio latente, mappano a una distribuzione (tipicamente gaussiana). Questo consente:

- Di generare nuovi dati campionando da
- Di modellare incertezza e variabilità

10.10.1 Funzionamento

1. L'**encoder** genera i parametri di una distribuzione su uno spazio latente (tipicamente media e varianza di una gaussiana).
2. Si campiona da tale distribuzione per ottenere un vettore latente z .
3. Il **decoder** ricostruisce i dati originali a partire da z .
4. Si massimizza un **bound inferiore** sulla likelihood dei dati, noto come **ELBO** (Evidence Lower Bound).

10.10.2 Vantaggi

- Permettono di generare nuovi dati
- Forniscono una base probabilistica per il feature learning

10.10.3 Limiti

- I campioni generati tendono ad essere più sfocati rispetto ad altri modelli generativi (es. GAN)
- Massimizzano solo un bound della likelihood, non la likelihood stessa

10.11 Disentangled Variational Autoencoders (β -VAE)

I β -VAE sono un'estensione dei VAE progettata per imparare rappresentazioni latenti più interpretabili. Ogni dimensione dello spazio latente cerca di rappresentare un fattore di variazione indipendente (es. colore, rotazione, posa).

Obiettivo: ottenere uno **spazio latente strutturato**, in cui modificare una dimensione porta a un cambiamento interpretabile nella generazione del dato.

10.12 Ricerca Attiva su VAE

- Utilizzo di posteriori più complessi rispetto alla gaussiana diagonale
- Inserimento di strutture nei latenti per migliorare interpretabilità e performance

10.13 Generative Adversarial Networks (GAN)

I GAN abbandonano l'idea di modellare esplicitamente la densità dei dati e si concentrano sulla capacità di generare nuovi campioni. Il funzionamento si basa su un gioco a due giocatori:

- **Generatore**: prende rumore casuale e cerca di generare dati realistici
- **Discriminatore**: cerca di distinguere i dati reali da quelli generati

L'addestramento avviene tramite un gioco competitivo (minimax): il generatore cerca di ingannare il discriminatore, e il discriminatore cerca di non farsi ingannare.

10.13.1 Allenamento GAN

1. Si campiona rumore e si generano immagini finte tramite il generatore.
2. Si mostrano immagini reali e finte al discriminatore.
3. Si aggiorna il discriminatore per migliorare la sua capacità di distinguere tra reale e generato.
4. Si aggiorna il generatore per migliorare la sua capacità di ingannare il discriminatore.

In pratica si alternano gradient ascent sul discriminatore e gradient descent (o ascent) sul generatore.

Una variante standard dell'obiettivo del generatore è massimizzare la probabilità che il discriminatore creda che le immagini finte siano reali. Questo fornisce segnali di gradiente migliori.

10.13.2 Vantaggi

- Produzione di immagini di altissima qualità

10.13.3 Svantaggi

- Allenamento instabile e difficile da controllare
- Non è possibile calcolare la probabilità esplicita di un dato ()
- Difficile da usare per inferenza (es. stimare dato)

10.13.4 Aree di ricerca attiva

- Miglioramento delle funzioni di perdita (Wasserstein GAN, LSGAN)
- GAN condizionali (es. per generare immagini specifiche)

10.14 Riepilogo dei Modelli Generativi

- **PixelRNN / PixelCNN**: modello a densità esplicita, ottimizza la likelihood, genera buoni campioni ma lentamente
- **VAE**: modelli a densità esplicita che ottimizzano un bound inferiore della likelihood, utili per rappresentazioni latenti, generano dati meno nitidi
- **GAN**: modello implicito, usa approccio competitivo, campioni realistici ma addestramento instabile e nessuna inferenza possibile

—

11 Domain Adaptation

11.1 Cos'è il Domain Adaptation

I modelli di deep learning funzionano molto bene quando i dati in input hanno caratteristiche simili a quelli visti in fase di addestramento, ad esempio fotografie. Tuttavia, il loro rendimento cala drasticamente quando i dati cambiano dominio, ad esempio nel passaggio da foto a disegni o schizzi. Questo perché la distribuzione dei dati cambia, e i modelli non sono in grado di generalizzare efficacemente.

Le variazioni visive che causano problemi possono derivare da:

- Cambiamenti stagionali o temporali
- Condizioni di illuminazione
- Diversi sensori o dispositivi
- Costi elevati di raccolta dati reali

- Uso di dati sintetici

11.2 Il problema del Domain Adaptation

Si tratta di risolvere lo stesso compito (task) su due domini diversi: uno di partenza (source) e uno di arrivo (target). La difficoltà sta nel fatto che:

- I dati hanno distribuzioni diverse: $P(X_s) \neq P(X_t)$
- Le variabili input sono diverse: $X_s \neq X_t$
- Ma il task e l'output sono gli stessi: $T_s = T_t$, $Y_s = Y_t$

11.3 Unsupervised Domain Adaptation (UDA)

L'adattamento del dominio non supervisionato è il caso più comune. Qui il modello viene addestrato su un dominio di partenza **etichettato** e applicato a un dominio di destinazione **non etichettato**. L'obiettivo è ridurre la discrepanza tra i due domini per ottenere buone performance anche sul dominio di destinazione.

11.3.1 Transductive Setting

In questo scenario, il modello ha accesso durante l'addestramento sia ai dati etichettati del dominio di partenza che a quelli non etichettati del dominio di destinazione, ma verrà valutato solo sul target.

11.3.2 Varianti da considerare

1. Numero di sorgenti
 - Single-source**: un solo dominio sorgente
 - Multi-source**: più domini sorgente
 - Mixed**: i domini sorgente includono varianti eterogenee mescolate
2. Sovrapposizione delle etichette
 - Closed Set**: le etichette di source e target coincidono completamente
 - Partial**: il target ha solo un sottoinsieme delle etichette del source
 - Open Set**: il target contiene anche classi sconosciute (non presenti nel source)

11.4 Quando non si ha accesso ai dati target

Due strategie comuni:

1. Domain Generalization

Il modello viene addestrato su più domini sorgente per apprendere una rappresentazione generalizzata che funzioni bene anche su domini non visti, senza adattamento esplicito.
2. Continuous Domain Adaptation

L'adattamento avviene in modo progressivo, attraversando una sequenza di domini intermedi. Questo consente un allineamento graduale verso il dominio di destinazione.

Differenza principale:

- Domain Generalization mira alla robustezza verso domini ignoti
- Continuous Domain Adaptation si concentra sull'allineamento incrementale

11.5 Algoritmi Shallow per Domain Adaptation

11.5.1 Transfer Component Analysis (TCA)

TCA apprende un mapping kernel di bassa dimensionalità che proietta i dati in uno spazio in cui source e target risultino più simili. Il mapping è definito da una funzione kernel .

11.5.2 Maximum Mean Discrepancy (MMD)

MMD misura la distanza tra le distribuzioni di source e target nello spazio di Hilbert riproducibile (RKHS). L'obiettivo è minimizzare questa distanza per allineare le distribuzioni.

11.6 Adversarial Domain Adaptation – RevGrad

11.6.1 RevGrad

RevGrad introduce un meccanismo di **inversione del gradiente** (Gradient Reversal Layer, GRL) per rendere le rappresentazioni apprese indistinguibili tra source e target.

- Durante il **forward pass**: il GRL non fa nulla.
- Durante il **backward pass**: inverte i gradienti (moltiplicandoli per -1), inducendo il modello a imparare rappresentazioni confuse per il discriminatore.

11.6.2 Architettura RevGrad-DANN

- **Feature Extractor**: estrae rappresentazioni dai dati
- **Label Predictor**: predice l'etichetta del task principale
- **Domain Discriminator**: cerca di distinguere se una rappresentazione proviene da source o target

Il GRL collega il Feature Extractor al Domain Discriminator e promuove **rappresentazioni invarianti al dominio**.

Obiettivo: Generalizzare dal dominio source al dominio target riducendo le differenze tra le loro distribuzioni. Le rappresentazioni condivise rendono il modello più robusto e adattabile.

11.7 Altre Varianti Adversariali

11.7.1 Asymmetric Adversarial Domain Adaptation

Usa architetture diverse o meccanismi di perdita asimmetrici per adattare source e target, permettendo rappresentazioni più flessibili o specifiche per dominio.

11.7.2 ADDA – Adversarial Discriminative Domain Adaptation

L'ADDA si svolge in tre fasi:

1. **Pre-training**

Si usa un dominio source con etichette

Le immagini vengono elaborate per imparare rappresentazioni

2. **Adversarial Adaptation**

Si introduce il dominio target (senza etichette)

Si usa un discriminatore per distinguere tra source e target

Il target CNN cerca di ingannare il discriminatore, imparando rappresentazioni simili a quelle del source

3. **Testing**

Una volta addestrato, il modello target viene usato per classificare immagini target

11.8 Benchmark per Domain Adaptation

- **PACS**

Valuta la generalizzazione verso domini stilizzati e artistici (Photo, Art, Cartoon, Sketch)

- **Office-Home**

Domini realistici e stilizzati (es. oggetti d'ufficio, decorazioni, ecc.)

- **DomainNet**

Dataset su larga scala con domini visivamente eterogenei (Clipart, Real, Sketch, Painting, Quick-draw).

Questi dataset permettono di valutare la performance dei modelli in presenza di **distribution shifts** tra dati di addestramento e dati di test.

Un esempio emblematico riguarda la **Semantic Segmentation**:

- C'è un **ampio divario visivo** tra immagini sintetiche e reali (source vs. target)
- Tuttavia, la **struttura spaziale** resta simile (strade, alberi, veicoli), quindi la sfida principale è ridurre la discrepanza visiva mantenendo l'allineamento spaziale

11.9 Pixel-Level Domain Adaptation e GANs

Quando la differenza visiva tra source e target è elevata (es. immagini sintetiche vs reali), si può intervenire a livello **pixel** con modelli generativi:

- GAN per Pixel-Level D

Le immagini source vengono trasformate per assomigliare visivamente al target, mantenendo però la struttura spaziale (utile per segmentazione e detection)

- Cycle-GAN

Traduce immagini senza necessità di coppie accoppiate. Traduce source $\rightarrow$ target e poi ricostruisce target $\rightarrow$ source per garantire **coerenza ciclica**.

- SBADA-GAN

Un'estensione del Cycle-GAN che combina coerenza semantica e traduzione visiva. Traduce le immagini in target e poi le riporta in source, preservando il significato.

- Pixel-Level DA in Detection

Nel contesto dell'object detection, il pixel-level DA è cruciale per trasformare immagini sintetiche in realistiche. Riducendo il **visual gap**, i modelli possono generalizzare meglio anche su dati visivamente differenti.

11.10 Batch Normalization per Domain Adaptation

La **Batch Normalization (BN)** è utile per normalizzare le attivazioni delle reti neurali. Nel DA, può aiutare a compensare le differenze tra source e target:

- BN adatta le statistiche alle distribuzioni del target
- **Multi-Source BN**: combina statistiche da più domini sorgente
- **Larger Norm More Transferable**: si usa BN e una loss sulla norma per migliorare la trasferibilità

In particolare:

- **BN per DA:** consente al modello di mantenere stabilità durante il training anche se source e target sono molto diversi.
- **Multi-Source BN:** calcola statistiche di normalizzazione congiunte da più source per migliorare l'adattamento.
- **Larger Norm:** l'uso di una norma più ampia favorisce trasferibilità e riduzione della distanza tra le distribuzioni.

11.11 Domain Generalization

Si allena il modello su più domini per renderlo **robusto a domini non visti** (senza usarli durante il training). Tecniche comuni:

- Self-Challenging for DG

Crea versioni più difficili dei dati di training per forzare il modello a imparare feature robuste, simulando condizioni di dominio perturbate.

- Jigsaw Puzzle-Based DG

Spezza e rimescola le immagini in patch: il modello deve ricomporle, focalizzandosi su **contenuto semantico** e **struttura globale** più che su dettagli specifici di un dominio. Questo approccio incoraggia rappresentazioni **domain-agnostic** e migliora la generalizzazione a domini non visti.

Questi approcci sono fondamentali per affrontare il **domain shift**, ovvero il disallineamento tra i dati di addestramento e quelli di test, che è una sfida cruciale per l'uso del machine learning nel mondo reale.